INTERVIEW PREPARATION GUIDE

AWS for Senior Full-Stack Developers

React + Node.js on AWS: Lambda, API Gateway, AppSync, Aurora MySQL, networking, CloudFormation, messaging, monitoring and system design, with diagrams, code, model answers and scenarios.

- Fundamentals
- Lambda and serverless
- API Gateway and AppSync
- Databases
- Networking and security
- CloudFormation
- S3, SQS, SNS
- Monitoring
- System design

Contents

1. Learning roadmap and reference architecture
2. Module 1: AWS fundamentals and architecture
3. Module 2: AWS Lambda and serverless
4. Module 3: API Gateway and AppSync
5. Module 4: Databases (RDS, Aurora MySQL, MongoDB)
6. Module 5: Networking and security
7. Module 6: CloudFormation and deployment
8. Module 7: Storage, messaging and asynchronous processing
9. Module 8: Monitoring and troubleshooting
10. Module 9: System design and real-world scenarios
11. Appendix: cheat sheets, glossary, weak-topic tracker

Each module follows the same ten-step format: simple explanation, analogy, architecture diagram, practical example, request flow, production considerations, common mistakes, interview questions (beginner, intermediate, senior), scenario questions and a practice exercise.

Learning roadmap

You already build React and Node.js applications. Senior interviews test whether you can **run** them on AWS: secure, scalable, observable and recoverable. This guide follows the order in which the pieces depend on each other: first the map of services, then compute and APIs, then data, then the network and security that wrap them, then deployment, messaging, operations and finally system design.

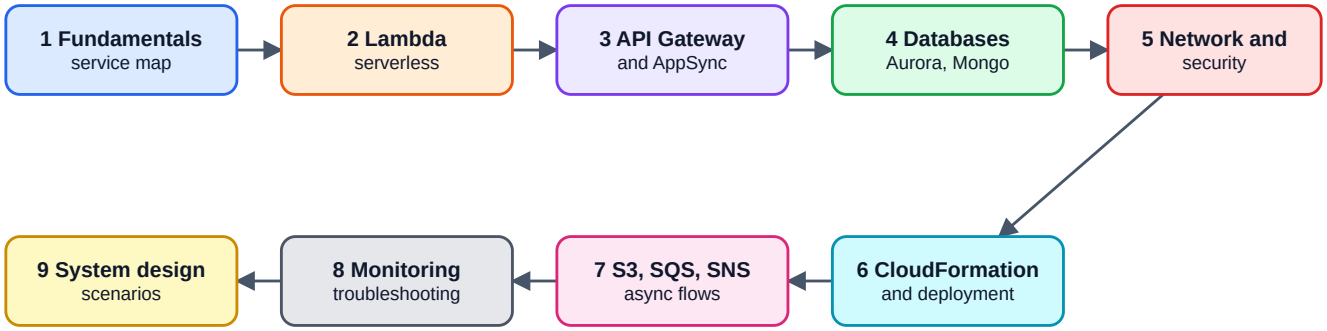


Figure 1. The nine-module roadmap. Each module ends with interview questions, scenarios and a practice task.

Suggested 4-week plan

Week	Focus	Hands-on goal
1	Modules 1 to 3	Deploy a Lambda behind API Gateway; add an AppSync API reading from the same data
2	Modules 4 and 5	Aurora MySQL in private subnets, Lambda reaches it through RDS Proxy with Secrets Manager
3	Modules 6 and 7	Whole stack in one CloudFormation template; S3 presigned upload; SQS with DLQ
4	Modules 8 and 9	Dashboards and alarms; rehearse the seven system-design scenarios out loud

How to use this PDF with the interactive mode

- Read a module, then close the PDF and explain it out loud as if to an interviewer.
- Ask Claude: **"Quiz me on Module N"**. It asks one question at a time, waits for your answer, scores correctness, completeness and clarity, then asks a follow-up.
- Use the weak-topic tracker at the end of this PDF to log what you missed and revisit it.

> **Accuracy note**
>
> AWS limits and prices change. Numbers here reflect current documented defaults at the time of writing; confirm quotas in the Service Quotas console before quoting them as facts in a design review.

MODULE 1

AWS fundamentals and architecture

1. Simple explanation

AWS is a catalogue of building blocks that you rent by usage. For a React + Node.js product you need the same layers as anywhere: **somewhere to serve the frontend**, **an API**, **compute for your business logic**, **databases**, **background processing**, and **security plus monitoring** around them. The interview skill is knowing which AWS service fills each layer and why you chose it over the alternatives.

Key terms: a **Region** is a geographic area (for example ap-south-1); an **Availability Zone (AZ)** is one or more data centres inside a Region, isolated from failures in other AZs; **managed service** means AWS operates the servers, patching and often scaling for you.

2. Real-world analogy

> **Analogy**
>
> Building an app on AWS is like opening a restaurant in a food court. The landlord (AWS) provides electricity, plumbing, security and the building (data centres, hardware, network). You choose the kitchen setup: a ready-made kitchen with staff you pay per dish (Lambda), a rented full kitchen you operate (ECS/EC2), a delivery counter (CloudFront), a storeroom (S3), and a ledger (database). You are still responsible for the recipes, who gets keys to the kitchen, and what is in the food: this is the **shared responsibility model**.

3. Architecture diagram

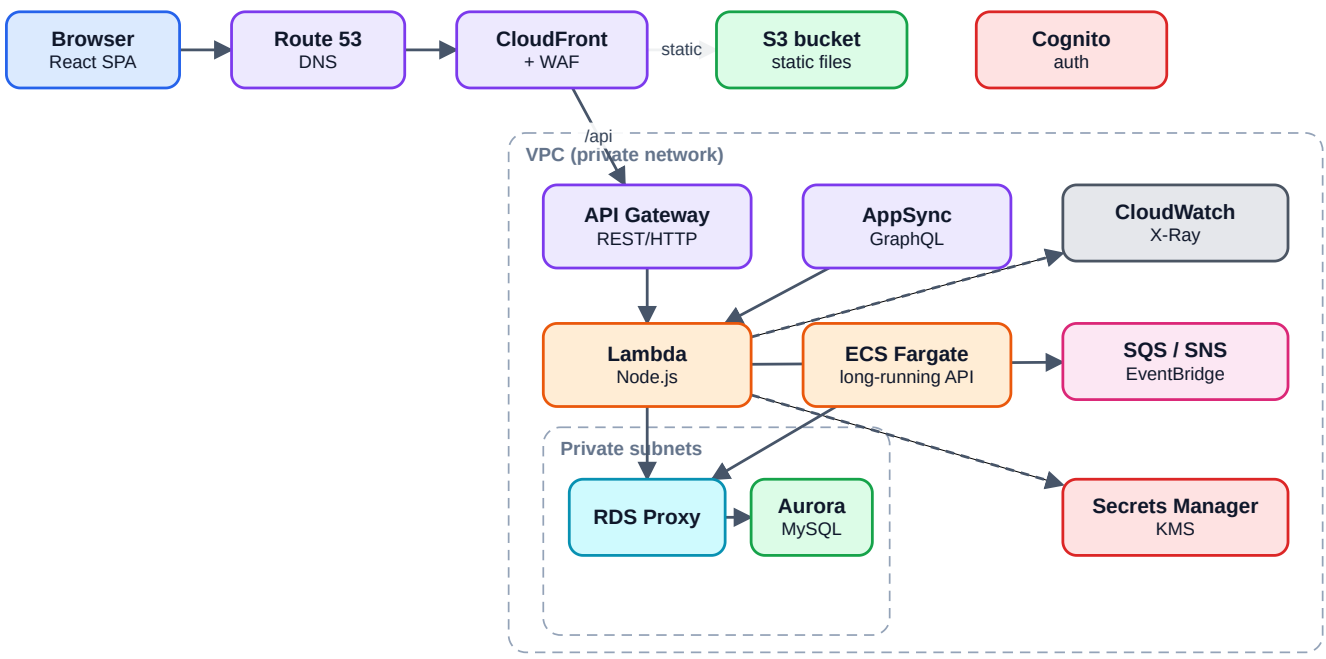


Figure 2. Reference architecture for a React + Node.js product on AWS.

4. Practical example

The mapping most senior interviewers expect you to produce on a whiteboard:

Layer	AWS service	Choose it when	Alternative

Static hosting + CDN	S3 + CloudFront (Origin Access Control)	SPA built with Vite/CRA; global low latency	Amplify Hosting (less control), Vercel
DNS and certificates	Route 53, ACM	Custom domain, free TLS certs	External DNS
Auth	Cognito (or Auth0/OIDC)	Sign-up, JWT, social login	Self-built (avoid)
API front door	API Gateway (REST/HTTP), AppSync, ALB	Throttling, authorizers, GraphQL	Express on ECS behind ALB
Compute	Lambda, ECS Fargate, EC2	Lambda: spiky/event driven. Fargate: steady, long-lived, WebSockets	Elastic Beanstalk, App Runner (check availability for new accounts)
Relational DB	Aurora MySQL, RDS MySQL	Transactions, joins, existing MySQL skills	Aurora PostgreSQL
Document DB	MongoDB Atlas on AWS, DocumentDB	Flexible schema; Atlas for full MongoDB features	DynamoDB (different model)
Async	SQS, SNS, EventBridge, Step Functions	Decouple, retry, fan-out, orchestrate	Kafka/MSK for streams
Secrets and keys	Secrets Manager, SSM Parameter Store, KMS	Rotating DB credentials, encryption	Env vars (not for secrets)
Observability	CloudWatch, X-Ray, CloudTrail	Logs, metrics, alarms, traces, audit	Datadog, New Relic
IaC and CI/CD	CloudFormation/SAM/CDK, CodePipeline, GitHub Actions	Repeatable environments	Terraform

Lambda vs ECS Fargate vs EC2

	Lambda	ECS Fargate	EC2
Unit of scale	Request (one environment per concurrent request)	Task (container)	Instance
Max run time	15 minutes	Unlimited	Unlimited
Idle cost	Zero	Pay while tasks run	Pay while instance runs
Ops effort	Lowest	Low	Highest (patching, AMIs)
Cold start	Yes	Task start takes seconds to minutes; keep min tasks	No, if running
Best for	Event glue, bursty APIs, cron	Steady APIs, WebSockets, heavy dependencies	Special hardware, licensing, full control

5. Step-by-step request flow

1. User opens app.example.com. Route 53 resolves it to a CloudFront distribution.
2. CloudFront serves index.html and JS bundles from its edge cache; on a miss it fetches from the private S3 bucket using Origin Access Control.
3. React boots, redirects to Cognito for login, receives a JWT.
4. React calls /api/orders with the JWT. CloudFront forwards /api/* to API Gateway (or the browser calls API Gateway/AppSync directly with CORS).
5. API Gateway validates the JWT, applies throttling, invokes Lambda.
6. Lambda reads the DB password from Secrets Manager (cached), connects through RDS Proxy to Aurora and returns JSON.
7. Logs, metrics and the X-Ray trace land in CloudWatch; alarms notify the team.

6. Production considerations

- **Security:** private subnets for data, least-privilege IAM, WAF in front, TLS everywhere, no public S3 buckets.
- **Scalability:** pick services that scale horizontally on their own (S3, CloudFront, Lambda, SQS, Aurora readers); remember the database is the usual bottleneck.

- **Reliability:** design for at least two AZs; define RPO/RTO; backups and tested restores.
- **Cost:** NAT gateways, data transfer, idle provisioned capacity and verbose logs are the usual surprises.
- **Operations:** everything defined as code; one pipeline promotes the same artifact through dev, staging and prod.

7. Common mistakes

Watch out

- Putting the database in a public subnet or opening 0.0.0.0/0 on port 3306.
- Using one AWS account for dev, staging and production instead of separate accounts with AWS Organizations.
- Treating Lambda as free scaling for every workload, including steady high-throughput APIs where containers are cheaper.
- Hard-coding Region, account ids and ARNs rather than using parameters.
- Skipping tagging, budgets and alarms, so the first sign of trouble is the bill.

8. Interview questions

BEGINNER

What is the difference between a Region and an Availability Zone?

Sample answer. A Region is a geographic area with multiple, isolated AZs. An AZ is one or more data centres with independent power and networking. I deploy across at least two AZs for high availability and across Regions only when I need disaster recovery or latency for a global audience.

BEGINNER

How would you host a React single-page app on AWS?

Sample answer. Build to static files, store them in a private S3 bucket and serve through CloudFront using Origin Access Control, with an ACM certificate and Route 53 record. For SPA routing I map 403/404 to index.html, set long cache headers on hashed assets and short cache on index.html, and deploy by syncing to S3 then invalidating index.html.

INTERMEDIATE

Explain the shared responsibility model.

Sample answer. AWS secures the cloud itself: facilities, hardware, hypervisor, and the managed service internals. I secure what I put in it: IAM policies, network rules, data encryption choices, OS patching on EC2, application code and configuration. With managed services such as Lambda or RDS, AWS takes more of the stack, but IAM, data and configuration remain mine.

INTERMEDIATE

When would you choose ECS Fargate over Lambda?

Sample answer. When the workload is steady and high throughput, needs long-lived connections like WebSockets, runs longer than 15 minutes, has large dependencies or needs consistent latency without cold starts. Lambda wins for spiky or low traffic, event-driven glue and when I want zero idle cost and minimal operations.

SENIOR

How do you structure AWS accounts for a product team?

Sample answer. Use AWS Organizations with separate accounts for production, staging/dev, shared tooling and security/logging, with SCPs as guardrails, IAM Identity Center for human access and role assumption from CI. It limits blast radius, gives clean cost reporting and lets production have tighter controls.

SENIOR

What is your approach to cost control on AWS?

Sample answer. Tag everything, set budgets and anomaly detection, review the Cost Explorer top services monthly, and right-size. Typical wins: remove unused NAT gateways or use VPC endpoints, set log retention, use Graviton, use Savings Plans for steady compute, scale non-production down at night and use S3 lifecycle rules.

9. Scenario-based questions

SCENARIO

Your company is launching a customer portal with unpredictable traffic. Sketch the architecture and justify two key choices.

Strong answer. Frontend on S3+CloudFront; API Gateway + Lambda (scales to zero, absorbs spikes); Aurora Serverless v2 or Aurora with RDS Proxy because the data is relational; SQS for background work; Cognito for auth. Justifications: Lambda for unpredictable load without paying for idle capacity, and RDS Proxy plus concurrency limits because Lambda scale can overwhelm the database. I would mention the trade-offs (cold starts, connection management) before being asked.

SCENARIO

An auditor asks how production data is protected. What do you cover?

Strong answer. Network: DB in private subnets, security group allows only the app tier. Identity: roles with least privilege, no long-lived keys, MFA for humans. Encryption: KMS at rest for Aurora/S3/EBS, TLS in transit. Secrets in Secrets Manager with rotation. Audit: CloudTrail to a locked S3 bucket, Config rules, GuardDuty. Recovery: automated backups, PITR and a tested restore runbook.

10. Practice exercise

Exercise

Draw (on paper) the architecture for a "task manager" app: React frontend, Node.js API, MySQL database, email notifications and file attachments. Label each AWS service, every network boundary (public vs private), and mark where authentication happens. Then list **three things that could fail** and how the design copes. Practice explaining it in under 3 minutes.

Ready for live practice? Tell Claude: "Quiz me on Module 1". You will get one question at a time, feedback on your answer and follow-ups.

MODULE 2

AWS Lambda and serverless

1. Simple explanation

Lambda runs your function when an event happens and bills for the milliseconds it runs. Each running copy (an **execution environment**) handles **one request at a time**; more simultaneous requests means more environments. A **cold start** is the extra time to create a new environment and run your initialisation code; a **warm** invocation reuses one. Lambda scales compute, but **it does not scale your database**: thousands of concurrent environments can open thousands of connections.

Idempotent means running the same operation twice has the same effect as running it once. It matters because several Lambda paths can deliver an event more than once.

2. Real-world analogy

> **Analogy**
>
> Lambda is like hiring taxis instead of owning cars. A taxi (environment) takes one passenger (request) at a time. If ten people want a ride, ten taxis are dispatched. The first time, taxis need to drive to you (cold start); a taxi waiting at the rank is quicker (warm). The city (database) only has so many parking spaces: if 5,000 taxis arrive at once, they block the street. Reserved concurrency is a quota on the number of taxis you may send; provisioned concurrency is keeping some taxis idling outside with the engine on.

3. Architecture diagram

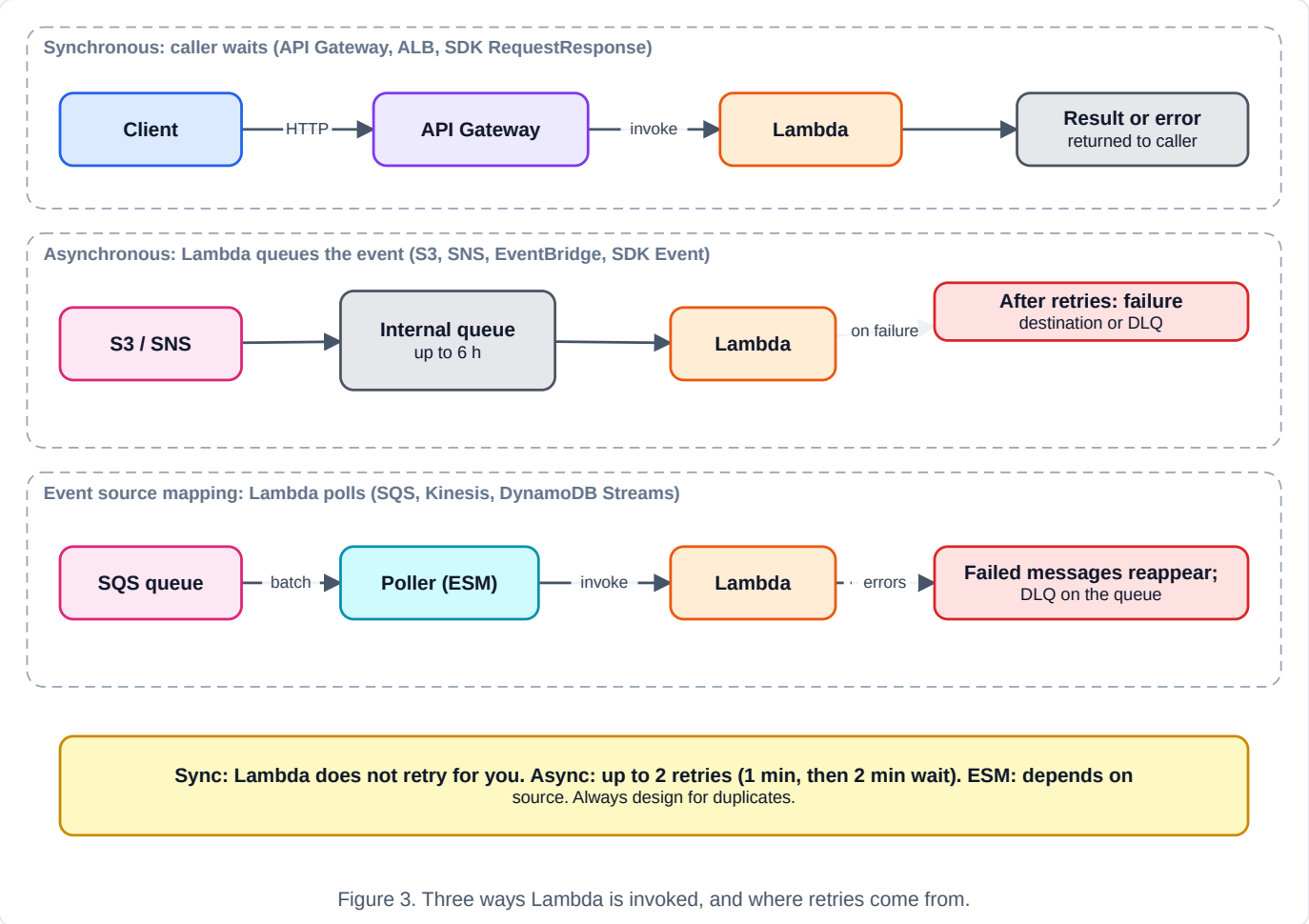


Figure 3. Three ways Lambda is invoked, and where retries come from.

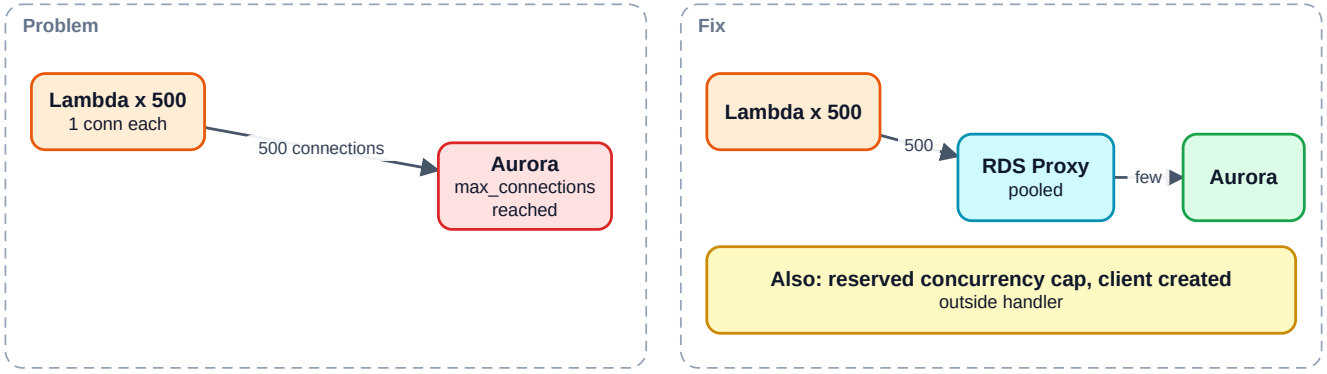


Figure 4. Connection storm and the RDS Proxy fix.

4. Practical example

Production-style handler (Node.js 22, mysql2, RDS Proxy, Secrets Manager). Note the client created *outside* the handler, the small pool, the partial-failure response and the idempotency guard.

```
import mysql from 'mysql2/promise';
import { SecretsManagerClient, GetSecretValueCommand } from '@aws-sdk/client-secrets-manager';

const sm = new SecretsManagerClient({});
let pool;                                   // reused while the environment is warm

async function getPool() {
  if (pool) return pool;
  const { SecretString } = await sm.send(new GetSecretValueCommand({ SecretId: process.env.DB_SECRET }));
  const s = JSON.parse(SecretString);
  pool = mysql.createPool({
    host: process.env.DB_PROXY_ENDPOINT,    // RDS Proxy endpoint, not the cluster endpoint
    user: s.username, password: s.password, database: 'shop',
    connectionLimit: 2,                     // one request at a time per environment
    ssl: { rejectUnauthorized: true }
  });
  return pool;
}

export const handler = async ({ Records }) => {          // SQS batch
  const db = await getPool();
  const batchItemFailures = [];
  for (const r of Records) {
    try {
      const o = JSON.parse(r.body);
      // idempotent insert: duplicate delivery becomes a no-op
      await db.execute('INSERT IGNORE INTO orders (id, user_id, total) VALUES (?, ?, ?)', [o.id, o.userId,
o.total]);
    } catch (e) {
      console.error(JSON.stringify({ msg: 'order failed', id: r.messageId, err: e.message }));
      batchItemFailures.push({ itemIdentifier: r.messageId });
    }
  }
  return { batchItemFailures };
};
```

Retry behaviour: precise reference

Invocation	Who retries	Details
Synchronous	The caller, not Lambda	Function error or throttle (429) is returned to the caller. API Gateway does not auto-retry; SDK clients may retry throttles/5xx. Make client retries use backoff.
Asynchronous	Lambda	Function errors: up to 2 retries (MaximumRetryAttempts 0 to 2); waits about 1 minute then 2 minutes. Throttles and service errors are retried with backoff up to MaximumEventAge (default 6 hours, configurable 60 s to 6 h). After that: on-failure destination/DLQ if configured, otherwise the event is dropped.

SQS event source mapping	The queue (visibility timeout)	If the function throws, the batch is not deleted and messages become visible again after the visibility timeout; after maxReceiveCount they go to the queue's DLQ. A Lambda-level DLQ setting does not apply here. Use partial batch responses. Set visibility timeout to at least 6 x function timeout.
Kinesis / DynamoDB Streams	Lambda, in order per shard	A failing batch is retried until it succeeds or the record expires, blocking that shard unless you set maximum retry attempts, maximum record age, bisect batch on error and an on-failure destination.

Not every failure triggers a retry. A sync caller that never retries simply fails; an async event that exceeds the max event age is dropped without a destination; and a code bug that returns success after catching an error hides the failure entirely.

Concurrency, timeouts and configuration

Setting	What it does	Interview note
Reserved concurrency	Guarantees and caps a function's share of the account limit (no charge)	Protects the database and other functions. Set to 0 to disable quickly
Provisioned concurrency	Pre-initialised environments (billed)	Removes cold starts on a published version/alias; can be scheduled
Timeout (max 15 min)	Hard stop for one invocation	Set lower than upstream timeouts; API Gateway default is about 29 s
Memory 128 MB to 10,240 MB	Also scales CPU	Test with Lambda Power Tuning
Environment variables	Config, up to 4 KB total	Not for secrets; store a secret ARN and fetch at init
Execution role	Permissions the function uses	Least privilege per function
Resource-based policy	Who may invoke the function	Use SourceArn to avoid confused deputy

5. Step-by-step request flow

1. API Gateway receives POST /orders and invokes Lambda synchronously.
2. Lambda finds an idle environment (warm) or creates one (cold: download code, start runtime, run top-level code including the pool setup).
3. The handler validates input and writes the order to the database through RDS Proxy.
4. For slow follow-up work (email, invoice PDF) the handler sends a message to SQS and returns 202 immediately.
5. A second Lambda polls SQS; on partial failure only failed messages are retried; repeated failures land in the DLQ.
6. Duration, Errors, Throttles and ConcurrentExecutions are published to CloudWatch; an alarm watches the DLQ depth.

6. Production considerations

- **Security:** one role per function, scoped resources, secrets from Secrets Manager, VPC only when the function needs private resources.
- **Scalability:** concurrency = requests per second x average seconds. Cap it so downstream systems are not overwhelmed; scaling is limited by account quota and a per-function scale-up rate.
- **Performance:** arm64, small bundle (esbuild), lazy-load, clients outside the handler, tune memory; SnapStart or provisioned concurrency only where latency demands it.
- **Reliability:** idempotent handlers, timeouts on every downstream call shorter than the function timeout, DLQs and alarms.
- **Cost:** duration x memory + requests; logs and NAT often cost more than compute.

7. Common mistakes

Watch out

- Creating the DB connection inside the handler so every request reconnects.
- Assuming Lambda automatically fixes database scaling, or that every failure is retried.
- Catching an error, logging it and returning success, so no retry or DLQ ever happens.

- SQS visibility timeout shorter than the function timeout, causing duplicates.
- Putting secrets in plain environment variables or committing them.
- Calling Lambda from Lambda synchronously and paying twice for waiting.
- Ignoring throttles: they are not counted as Errors.

8. Interview questions

BEGINNER

What is a cold start and how do you reduce it?

Sample answer. A cold start is the time to create a new execution environment and run init code. I reduce it with small bundles, lazy loading, arm64, creating SDK clients once, SnapStart for supported runtimes and provisioned concurrency for latency-critical paths.

BEGINNER

What are the limits you should remember for Lambda?

Sample answer. 15 minute timeout, 128 MB to 10 GB memory, 6 MB payload for synchronous calls, 4 KB of environment variables, 50 MB zip upload / 250 MB unzipped, 10 GB container images, and a default account concurrency of 1,000 per Region that can be raised.

INTERMEDIATE

Reserved vs provisioned concurrency?

Sample answer. Reserved concurrency reserves part of the account limit for a function and also caps it, at no cost, so it protects downstream systems and neighbours. Provisioned concurrency keeps pre-initialised environments ready to avoid cold starts and is billed. They solve different problems.

INTERMEDIATE

Explain retry behaviour for sync, async and SQS invocations.

Sample answer. Sync: Lambda returns the error to the caller and does not retry; the caller decides. Async: Lambda retries function errors twice and retries throttles for up to six hours, then uses the failure destination or DLQ. SQS: the message reappears after the visibility timeout and moves to the queue DLQ after maxReceiveCount; I use partial batch responses so only failed messages repeat.

INTERMEDIATE

How do you make a Lambda idempotent?

Sample answer. Use a stable key such as the event or order id and enforce it at the data layer: a unique constraint with INSERT IGNORE / ON DUPLICATE KEY, or a DynamoDB conditional write with a TTL. Powertools has an idempotency utility. I do not rely on in-memory sets because environments are ephemeral.

SENIOR

Lambda opens too many MySQL connections. What do you do?

Sample answer. Create the pool outside the handler with a tiny connection limit, put RDS Proxy in front of Aurora so connections are multiplexed, cap the function with reserved concurrency or SQS maximum concurrency, keep queries short, and monitor DatabaseConnections. If the access pattern allows, move hot key-value reads to DynamoDB. Lambda scales compute, not the database.

SENIOR

Your Lambda times out sporadically. How do you investigate?

Sample answer. Check the REPORT lines for Duration and Init Duration, X-Ray subsegments for slow dependencies, DB wait times and connection acquisition, NAT/VPC routing, memory pressure and cold starts. Then add timeouts on downstream calls below the function timeout, tune memory, and fix the slow dependency; raising the timeout is the last resort.

9. Scenario-based questions

SCENARIO

Black Friday: orders spike 20x and your database CPU hits 100% with "too many connections". What now?

Strong answer. Immediate: lower reserved concurrency to protect the DB and put orders behind SQS so the write rate is controlled. Short term: RDS Proxy, a small pool per environment, add Aurora readers for read traffic, cache hot reads. Medium term: review slow queries and indexes, consider Aurora Serverless v2 scaling and partitioning heavy tables. Explain that SQS turns a spike into a smooth backlog.

SCENARIO

A payment webhook is delivered twice and a customer is charged twice. Why and how do you prevent it?

Strong answer. Providers retry webhooks (at-least-once), and Lambda or SQS may also duplicate delivery. Prevent with an idempotency key (provider event id) stored with a unique constraint in the same transaction as the business change, and return 200 for duplicates. Also use the provider idempotency key when calling the charge API.

10. Practice exercise

Exercise

Build a Lambda that consumes an SQS queue and inserts orders into MySQL. Requirements: pool outside the handler, `INSERT IGNORE` idempotency, partial batch response, a DLQ with maxReceiveCount 3, a CloudWatch alarm on the DLQ. Then send a poison message and a duplicate message and describe what each does in the logs.

Ready for live practice? Tell Claude: "Quiz me on Module 2". You will get one question at a time, feedback on your answer and follow-ups.

MODULE 3

API Gateway and AppSync

1. Simple explanation

API Gateway is the managed front door for REST/HTTP/WebSocket APIs: it authenticates, throttles, validates and routes requests to Lambda or HTTP backends. **AppSync** is the managed GraphQL service: clients send one query describing exactly the fields they want, and **resolvers** (small mapping functions) fetch data from **data sources** such as DynamoDB, Lambda, OpenSearch or HTTP endpoints. REST exposes many resources with fixed response shapes; GraphQL exposes one endpoint with a typed schema and client-shaped responses.

2. Real-world analogy

> **Analogy**
>
> API Gateway is a hotel reception desk: it checks your ID (authorizer), limits how many guests enter per minute (throttling), and sends you to the right department. AppSync is a concierge who takes one detailed shopping list (the GraphQL query), sends assistants to several shops (resolvers and data sources), and hands you a single bag with exactly what you asked for.

3. Architecture diagram

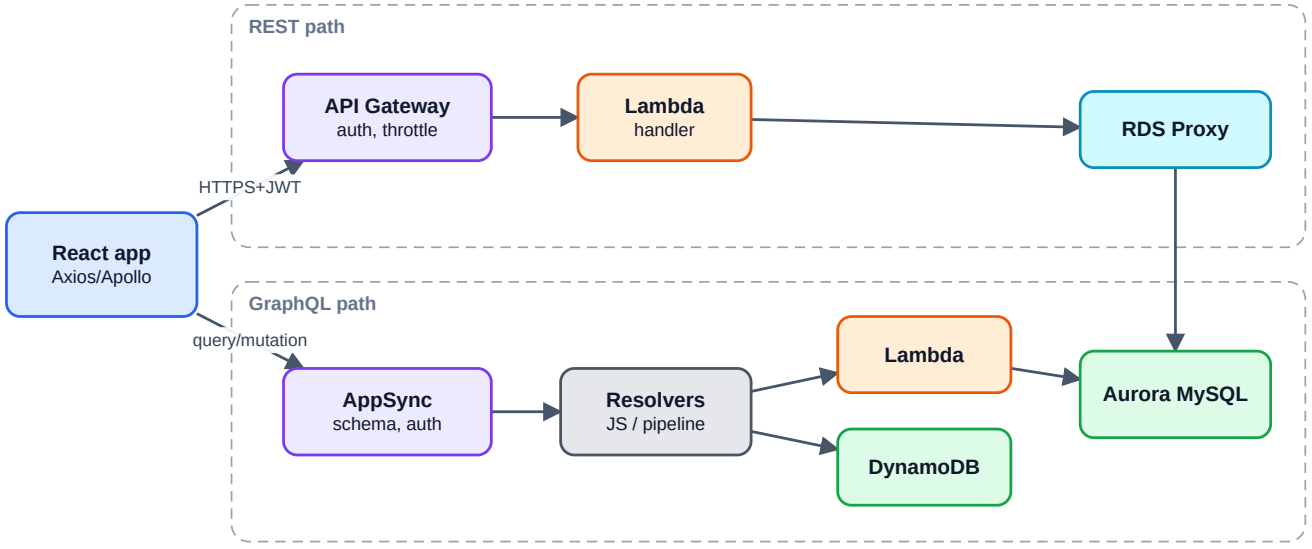


Figure 5. REST via API Gateway versus GraphQL via AppSync, both ending at the same data.

4. Practical example

HTTP API with a JWT authorizer, throttling and Lambda (SAM/CloudFormation):

```
Resources:
  Api:
    Type: AWS::Serverless::HttpApi
    Properties:
      Auth:
        Authorizers:
          Jwt:
            IdentitySource: $request.header.Authorization
            JwtConfiguration:
              issuer: !Sub https://cognito-idp.${AWS::Region}.amazonaws.com/${UserPool}
              audience: [ !Ref UserPoolClient ]
        DefaultAuthorizer: Jwt
```

```
      DefaultRouteSettings:
        ThrottlingBurstLimit: 200
        ThrottlingRateLimit: 100
  OrdersFn:
    Type: AWS::Serverless::Function
    Properties:
      Handler: orders.handler
      Runtime: nodejs22.x
      Events:
        Post: { Type: HttpApi, Properties: { ApiId: !Ref Api, Path: /orders, Method: POST } }
```

AppSync schema and JavaScript resolver (the resolver runs inside AppSync, no Lambda needed for DynamoDB):

```
# schema.graphql
type Order { id: ID!  total: Float!  items: [Item!]! }
type Query    { getOrder(id: ID!): Order }
type Mutation { createOrder(input: OrderInput!): Order }
type Subscription { onCreateOrder: Order @aws_subscribe(mutations: ["createOrder"]) }

// getOrder resolver (APPSYNC_JS)
import { util } from '@aws-appsync/utils';
export function request(ctx) {
  return { operation: 'GetItem', key: util.dynamodb.toMapValues({ pk: ctx.args.id }) };
}
export function response(ctx) { return ctx.result; }
```

Comparison

Topic	API Gateway REST	HTTP API	AppSync
Protocol	REST, WebSocket (separate)	REST (cheaper, faster)	GraphQL over HTTPS and WebSockets
Auth	IAM, Cognito, Lambda authorizer, API keys	IAM, JWT, Lambda authorizer	API key, Cognito, IAM, OIDC, Lambda authorizer
Throttling	Account + stage/method limits, usage plans	Route-level throttling	Request limits per API/account (check quotas)
Caching	Built-in stage cache (TTL default 300 s)	None built in (use CloudFront)	Per-resolver or full-request cache
Validation	Request models and parameters	Limited	Schema type validation
Real time	WebSocket API	No	Subscriptions (WebSockets) on mutations
Timeout	Integration default about 29 s	Up to 30 s	Resolver/request limits; long work async
Best for	Public REST APIs, usage plans, WAF	Simple low-cost APIs, JWT	Mobile/web apps needing flexible queries and real time

5. Step-by-step request flow

1. React sends a GraphQL query with the Cognito JWT to the AppSync endpoint.
2. AppSync validates the token and the schema, then runs the field resolvers. A pipeline resolver can run several functions in order.
3. A resolver calls a Lambda data source for Aurora MySQL (using RDS Proxy) and a DynamoDB data source for a cache table, in parallel.
4. AppSync assembles one JSON response with only the requested fields.
5. When a mutation runs, AppSync pushes the new data to clients subscribed over a WebSocket.
6. Logs and metrics go to CloudWatch; enable field-level logging and X-Ray when debugging.

6. Production considerations

- **Security:** never expose a public unauthenticated API; use JWT/IAM; field-level authorization (for example Cognito groups with @aws_auth); WAF for rate limiting and SQL injection rules; CORS limited to your origins.
- **Scalability:** throttle per client with usage plans (REST) or route limits; cache read-heavy responses; protect the database with reserved concurrency.

- **Performance:** avoid N+1 resolver patterns by batching (BatchGetItem, Lambda BatchInvoke or DataLoader), limit query depth and pagination size.
- **Reliability:** clients retry only idempotent calls with exponential backoff and jitter; return 429 with Retry-After.
- **Cost:** HTTP API is cheaper than REST; AppSync bills per operation and per real-time update; caches have hourly cost.

7. Common mistakes

Watch out

- Using API keys as authentication; they only identify/meter clients.
- Leaving CORS as * or forgetting CORS headers on Lambda proxy responses.
- Long-running work behind API Gateway: it times out near 29 s, so make it async (202 + status polling or subscription).
- GraphQL N+1: a list of 50 orders triggering 50 separate DB queries.
- No depth/complexity limits, letting one query pull the whole database.
- Choosing AppSync only because it sounds modern, when a simple REST API would do.

8. Interview questions

BEGINNER

REST vs GraphQL: when would you use each?

Sample answer. REST suits resource-oriented APIs, HTTP caching and simple clients. GraphQL suits UIs that need different shapes of data per screen, aggregating multiple sources in one round trip and real-time updates. The cost is more complexity: schema design, resolver performance, and authorization at field level.

BEGINNER

What does API Gateway do that a Lambda Function URL does not?

Sample answer. Authorizers, throttling and usage plans, request validation, caching on REST APIs, WAF integration, stages with canary releases, custom domains and detailed access logs. A Function URL is a simple HTTPS endpoint with IAM or no auth.

INTERMEDIATE

How do you authorize requests in API Gateway?

Sample answer. Cognito or JWT authorizers for user tokens, IAM (SigV4) for service-to-service, a Lambda authorizer for custom logic or third-party tokens, with caching of the authorizer result. Fine-grained rules like ownership checks still belong in the backend.

INTERMEDIATE

How do AppSync resolvers work, and what is a pipeline resolver?

Sample answer. A resolver maps a GraphQL field to a data source with a request function and response function (JavaScript runtime, or legacy VTL). A pipeline resolver chains several functions, for example authorize, fetch, then enrich, sharing context between them.

SENIOR

Throttling and caching: how do you protect a backend behind API Gateway?

Sample answer. Set stage and route throttles, usage plans per API key for partner clients, WAF rate-based rules for abusive IPs, enable the REST stage cache for idempotent GETs with correct cache keys, and cap downstream concurrency. I return 429 with Retry-After and make clients back off with jitter.

SENIOR

How would you optimise a slow GraphQL API backed by Aurora?

Sample answer. Trace which resolver is slow. Fix N+1 with batch resolvers or DataLoader, add the right composite indexes, select only needed columns, use keyset pagination, add read replicas for read-heavy queries, cache stable data in AppSync cache or Redis, limit depth and complexity, and consider denormalised read models in DynamoDB for hot screens.

9. Scenario-based questions

SCENARIO

Your REST endpoint returns 504 after 29 seconds when generating a large report. What do you change?

Strong answer. Make it asynchronous: POST creates a job and returns 202 with a job id; the work runs via SQS or Step Functions and Lambda/ECS; the result is written to S3; the client polls a status endpoint or receives an AppSync subscription / WebSocket push; deliver the file with a presigned URL.

SCENARIO

A partner is hammering your API and affecting other customers. Options?

Strong answer. Issue the partner an API key with a usage plan (rate, burst, daily quota) on a REST API, add a WAF rate-based rule, return 429 responses, and alarm on throttle counts. Long term, isolate heavy tenants (separate stage or queue) and contact the partner about backoff.

10. Practice exercise

Exercise

Create a Cognito user pool and an HTTP API with a JWT authorizer and one Lambda route `GET /me` that returns the user's `sub` claim. Then add an AppSync API with a `getOrder` query using a DynamoDB resolver and a subscription. Compare the number of round trips React needs for "order + items + customer" in each approach.

Ready for live practice? Tell Claude: "Quiz me on Module 3". You will get one question at a time, feedback on your answer and follow-ups.

MODULE 4

Databases: RDS, Aurora MySQL and MongoDB

1. Simple explanation

RDS runs managed relational databases (MySQL, PostgreSQL and others): AWS handles provisioning, patching, backups and Multi-AZ failover. **Aurora MySQL** is AWS's MySQL-compatible engine with a different storage layer: one cluster volume replicated six ways across three AZs, one writer instance and up to 15 reader instances that share the same storage. **MongoDB** on AWS means MongoDB Atlas (the vendor's managed service running on AWS), Amazon DocumentDB (an AWS service with a MongoDB-compatible API, not MongoDB itself) or a self-managed cluster on EC2.

Connection pooling keeps a small set of reusable database connections; **RDS Proxy** is a managed pooler between your app and RDS/Aurora.

2. Real-world analogy

> **Analogy**
>
> Aurora is a library with one librarian taking returns (the writer) and several librarians answering questions from the same shelves (readers). RDS Proxy is the front desk that queues visitors and gives them a small number of numbered tickets, so the library never has 5,000 people shouting at once. Backups are the nightly photocopy of the shelves plus a diary of every change, so you can rebuild the library as it was at 3:42 pm.

3. Architecture diagram

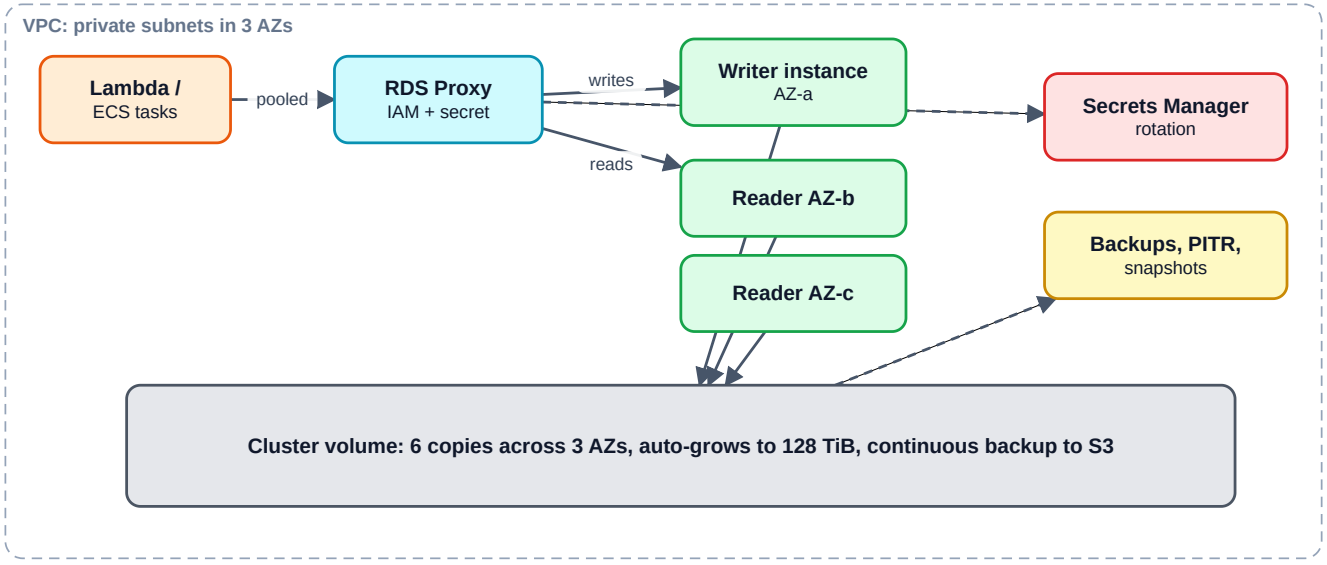


Figure 6. Aurora MySQL cluster with RDS Proxy, readers and shared storage across three AZs.

4. Practical example

Aurora MySQL cluster with RDS Proxy (CloudFormation excerpt):

```
DbCluster:
  Type: AWS::RDS::DBCluster
  DeletionPolicy: Snapshot
  Properties:
    Engine: aurora-mysql
    EngineVersion: 8.0.mysql_aurora.3.07.1        # check the current supported version
    ManageMasterUserPassword: true                # RDS creates and rotates the secret
    MasterUsername: admin
```

```
    DBSubnetGroupName: !Ref DbSubnets
    VpcSecurityGroupIds: [ !Ref DbSg ]
    StorageEncrypted: true
    BackupRetentionPeriod: 14
    DeletionProtection: true
Writer:
  Type: AWS::RDS::DBInstance
  Properties: { DBClusterIdentifier: !Ref DbCluster, Engine: aurora-mysql, DBInstanceClass: db.r6g.large }
Proxy:
  Type: AWS::RDS::DBProxy
  Properties:
    DBProxyName: shop-proxy
    EngineFamily: MYSQL
    RoleArn: !GetAtt ProxyRole.Arn
    VpcSubnetIds: [ !Ref PrivA, !Ref PrivB ]
    VpcSecurityGroupIds: [ !Ref ProxySg ]
    RequireTLS: true
    Auth: [ { AuthScheme: SECRETS, SecretArn: !GetAtt DbCluster.MasterUserSecret.SecretArn, IAMAuth: DISABLED } ]
```

Query performance habits (MySQL)

```
-- 1. Look at the plan
EXPLAIN ANALYZE SELECT id,total FROM orders WHERE user_id=42 ORDER BY created_at DESC LIMIT 20;
-- 2. Composite index: equality column first, sort column second
ALTER TABLE orders ADD INDEX idx_user_created (user_id, created_at);
-- 3. Keyset pagination instead of big OFFSET
SELECT id,total FROM orders WHERE (created_at,id) < (?,?) ORDER BY created_at DESC, id DESC LIMIT 20;
```

Options compared

Option	Strengths	Limits / notes
RDS MySQL Multi-AZ	Simple, standby for failover, familiar	Standby does not serve reads; storage scaling is less elastic
Aurora MySQL	Faster failover (typically well under a minute), up to 15 readers, fast clones, Global Database, Serverless v2 autoscaling ACUs	Costs more per instance; connection limits still apply
Aurora Serverless v2	Scales capacity in fine steps for variable load	Set min/max ACU; not free when idle at minimum
MongoDB Atlas on AWS	Full MongoDB features, replica sets/sharding, PrivateLink or VPC peering	Separate vendor billing; networking setup
Amazon DocumentDB	AWS-native, VPC-only, MongoDB-compatible API	Compatible subset: verify features/drivers you use
Self-managed MongoDB on EC2	Full control	You own patching, backups, failover: usually not worth it

5. Step-by-step request flow

1. A request arrives at Lambda, which already holds a pool created at init.
2. Lambda sends a query to the RDS Proxy endpoint; the proxy borrows a pooled connection to the Aurora writer (or the reader endpoint for read-only queries).
3. Aurora executes using indexes; writes go to the shared storage volume (quorum of 6 copies).
4. Result returns through the proxy to Lambda and then to the client.
5. If the writer fails, Aurora promotes a reader; the proxy reconnects, shortening the application-visible outage.
6. Backups run continuously; a bad deploy can be recovered by point-in-time restore into a NEW cluster.

6. Production considerations

- **Security:** private subnets, security group allows only the proxy/app security group on 3306, TLS required, encryption at rest with KMS, Secrets Manager rotation, IAM database authentication where practical, no public access.
- **Scalability:** scale reads with replicas and the reader endpoint; scale writes vertically or by sharding/CQRS; cache; use RDS Proxy to protect max_connections.

- **Performance:** indexes, EXPLAIN, avoid SELECT *, keyset pagination, connection reuse, Performance Insights.
- **Reliability:** Multi-AZ/readers, tested restores, deletion protection, snapshots before migrations, cross-Region replicas or Global Database for DR.
- **Cost:** right-size instances, Graviton (r6g/r7g), Serverless v2 for spiky workloads, delete old snapshots, watch I/O and data transfer.

7. Common mistakes

Watch out

- Treating replicas as backups: a bad DELETE replicates instantly.
- Connecting Lambda directly to the writer with large pools.
- Putting credentials in environment variables or source code.
- Never testing a restore; discovering the backup is unusable during an incident.
- Running big schema changes during peak hours without an online approach.
- Assuming DocumentDB equals MongoDB.
- Using the cluster endpoint for read-heavy reports and starving writes.

8. Interview questions

BEGINNER

What is the difference between RDS Multi-AZ and read replicas?

Sample answer. Multi-AZ is for availability: a synchronous standby that takes over on failure, and it does not serve traffic. Read replicas are for read scaling: asynchronous copies that serve reads and can lag. For Aurora, readers share the storage, so lag is typically small, and they double as failover targets.

BEGINNER

What backup options do you have?

Sample answer. Automated backups with point-in-time restore (retention up to 35 days), manual snapshots that persist until deleted, cross-Region snapshot copy, and AWS Backup for policy-driven schedules. Restores create a new cluster, so I plan RPO and RTO and practise the procedure.

INTERMEDIATE

Why use RDS Proxy with Lambda?

Sample answer. Each Lambda environment may open its own connection, so scale-outs can exhaust max_connections. The proxy multiplexes many application connections onto a smaller pool, handles failover more gracefully, and integrates with Secrets Manager and IAM auth. Caveat: session pinning (for example with temp tables or certain SET statements) reduces multiplexing, and it adds cost and a little latency.

INTERMEDIATE

How would you choose between Aurora MySQL and MongoDB?

Sample answer. If the data is relational with transactions, joins and reporting, Aurora MySQL. If documents with varying shapes dominate and the team knows Mongo, Atlas or DocumentDB. I would not pick on fashion; the access patterns and consistency needs decide, and polyglot is fine when each store has a clear job.

SENIOR

A table grows to 500M rows and queries slow down. What is your plan?

Sample answer. Check plans with EXPLAIN and Performance Insights; add or fix composite and covering indexes; remove unused indexes; switch to keyset pagination; archive old rows or partition by date; push analytics to a replica or a warehouse; cache hot reads; and review the schema (data types, JSON misuse). Scale instance size only after fixing the queries.

SENIOR

Explain your disaster recovery strategy for Aurora.

Sample answer. Define RPO/RTO first. In-Region: Multi-AZ via readers with automatic failover. Cross-Region: Aurora Global Database (low-second replication, managed failover) for tight RPO/RTO, or cross-Region snapshot copies for cheaper, slower recovery. Back up with PITR, protect with deletion protection and separate-account snapshot copies, and rehearse restore plus app cut-over.

9. Scenario-based questions

SCENARIO

After a deploy, the API shows "Too many connections" and latency spikes. Walk me through diagnosis and fix.

Strong answer. Check DatabaseConnections and Lambda ConcurrentExecutions in CloudWatch, and whether the deploy removed pooling or created clients inside the handler. Stabilise: reduce concurrency, roll back the release if needed. Fix: pool outside handler with small limit, RDS Proxy, SQS buffering for writes, closing leaked connections, and alarms on connections and proxy pinning.

SCENARIO

A developer ran DELETE without WHERE at 14:05. How do you recover?

Strong answer. Do not touch the live cluster further. Use point-in-time restore to 14:04 into a new cluster (or Aurora backtrack if enabled), validate, then either swap the endpoint or copy the missing rows back into production. Afterwards add safeguards: least-privilege DB users, review for destructive SQL, and deletion protection.

10. Practice exercise

Exercise

Create an Aurora MySQL cluster (or run a local MySQL) with an `orders` table of 1M generated rows. Run the "latest 20 orders for a user" query before and after adding the composite index and compare `EXPLAIN ANALYZE`. Then rewrite the pagination with a keyset cursor and write down why OFFSET degrades.

Ready for live practice? Tell Claude: "Quiz me on Module 4". You will get one question at a time, feedback on your answer and follow-ups.

MODULE 5

AWS networking and security

1. Simple explanation

A **VPC** is your private network in AWS. You divide it into **subnets** (one per AZ per tier). A subnet is **public** if its route table sends 0.0.0.0/0 to an **Internet Gateway (IGW)**; it is **private** if it has no such route. Private resources reach the internet outward through a **NAT gateway** that lives in a public subnet. **Security groups** are stateful firewalls attached to resources; **network ACLs** are stateless allow/deny lists on subnets. **IAM** controls who can call AWS APIs; **Secrets Manager** stores and rotates credentials.

2. Real-world analogy

Analogy

The VPC is an office building. The public subnet is the lobby with a street door (IGW) and a reception desk. Private subnets are the back offices and the server room, with no street door. The NAT gateway is the mailroom: staff can send letters out and receive replies, but strangers cannot walk in. Security groups are the badge readers on each room (only listed badges enter); NACLs are the security guard at each corridor with a written blocklist; IAM is the HR system defining what each employee may do; Secrets Manager is the key safe that changes the combination regularly.

3. Architecture diagram

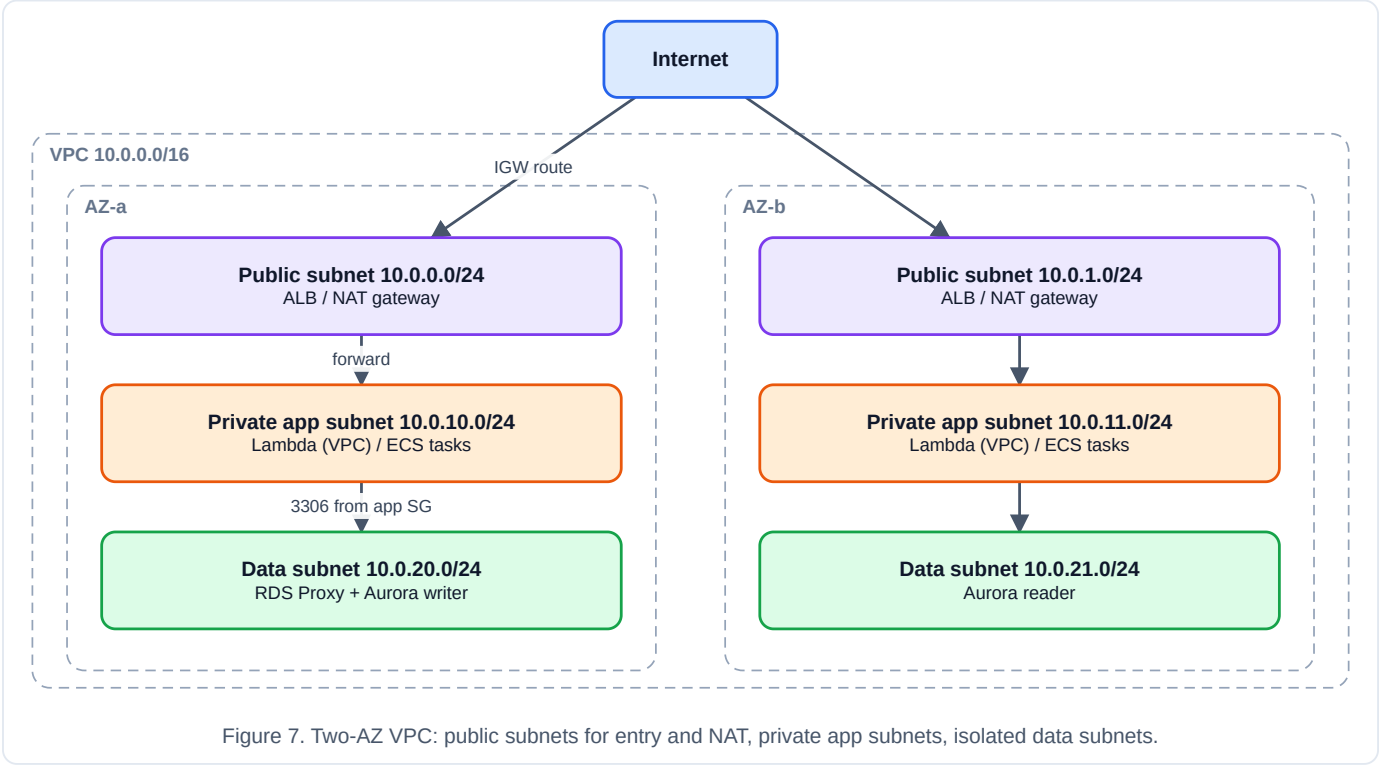


Figure 7. Two-AZ VPC: public subnets for entry and NAT, private app subnets, isolated data subnets.

4. Practical example

Security groups that chain by reference (CloudFormation):

```
AppSg:
  Type: AWS::EC2::SecurityGroup
  Properties: { GroupDescription: App tier, VpcId: !Ref Vpc }
ProxySg:
  Type: AWS::EC2::SecurityGroup
  Properties:
```

```
    GroupDescription: RDS Proxy
    VpcId: !Ref Vpc
    SecurityGroupIngress:
      - { IpProtocol: tcp, FromPort: 3306, ToPort: 3306, SourceSecurityGroupId: !Ref AppSg }
DbSg:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: Aurora
    VpcId: !Ref Vpc
    SecurityGroupIngress:
      - { IpProtocol: tcp, FromPort: 3306, ToPort: 3306, SourceSecurityGroupId: !Ref ProxySg }
```

Least-privilege IAM policy for a Lambda that reads one secret and writes to one queue:

```
{ "Version": "2012-10-17",
  "Statement": [
   { "Effect": "Allow", "Action": "secretsmanager:GetSecretValue",
     "Resource": "arn:aws:secretsmanager:ap-south-1:111122223333:secret:prod/db-*" },
   { "Effect": "Allow", "Action": "sqs:SendMessage",
     "Resource": "arn:aws:sqs:ap-south-1:111122223333:orders-queue" } ] }
```

Security groups vs NACLs

	Security group	Network ACL
Level	Instance / ENI (also Lambda, RDS, ALB)	Subnet
State	Stateful (return traffic automatic)	Stateless (allow both directions)
Rules	Allow only	Allow and deny, evaluated by rule number
Default	Deny inbound, allow outbound	Default NACL allows all
Use for	Primary control; reference other SGs	Coarse guardrail, blocking a bad CIDR

How a VPC Lambda reaches things

Destination	Path	Notes
Aurora / RDS Proxy	Local VPC route + security groups	No internet needed
S3, DynamoDB	Gateway VPC endpoint (free)	Avoids NAT data charges
Secrets Manager, SQS, KMS	Interface VPC endpoints (PrivateLink) or NAT	Endpoints cost per hour per AZ; NAT costs per hour plus per GB
Third-party API	NAT gateway in a public subnet	One NAT per AZ for resilience; without it the function has no internet

5. Step-by-step request flow

1. Lambda is configured with private subnets in two AZs and the app security group; Lambda creates shared network interfaces (ENIs) in those subnets.
2. On cold start it calls Secrets Manager through an interface endpoint (or NAT) using its execution role.
3. It opens TLS to the RDS Proxy endpoint. The route table sends the traffic locally; the proxy security group allows port 3306 from the app SG.
4. The proxy authenticates to Aurora with the stored credentials; the DB security group allows only the proxy SG.
5. Return traffic flows back automatically (security groups are stateful).
6. CloudTrail records the GetSecretValue call; VPC Flow Logs can show accepted/rejected traffic when debugging.

6. Production considerations

- **Security:** data tier has no route to an IGW; reference security groups instead of CIDRs; separate roles per workload; encryption in transit and at rest; GuardDuty, CloudTrail, Config; rotate secrets.
- **Scalability:** size CIDR ranges generously (you cannot easily shrink later); keep spare IPs for scaling tasks.
- **Performance:** VPC endpoints keep traffic on the AWS network; keep app and DB in the same Region and ideally same AZ for hot paths.

- **Reliability:** two or three AZs; one NAT gateway per AZ for production.
- **Cost:** NAT gateway data processing is a common bill shock; use gateway endpoints for S3/DynamoDB and consider interface endpoints for chatty AWS APIs.

7. Common mistakes

Watch out

- Putting Lambda in a VPC without a NAT or endpoints, then wondering why the SDK call hangs.
- Opening 3306 to 0.0.0.0/0 "temporarily".
- Using NACLs as the main control and forgetting ephemeral return ports.
- Using the administrator role for everything, or Action: * on Resource: *.
- Storing secrets in environment variables, repositories or CloudFormation parameters in plain text.
- Single NAT gateway for all AZs, so one AZ failure breaks outbound traffic.
- Confusing authentication (who are you) with authorization (what may you do).

8. Interview questions

BEGINNER

What makes a subnet public?

Sample answer. Its route table has a route to an Internet Gateway for 0.0.0.0/0 (and resources have public IPs). A private subnet has no such route; it may route outbound traffic to a NAT gateway.

BEGINNER

Security group vs NACL?

Sample answer. Security groups are stateful, allow-only firewalls attached to resources. NACLs are stateless, ordered allow/deny rules on a subnet. I rely mainly on security groups and use NACLs for broad blocks.

INTERMEDIATE

How does a Lambda in a private subnet call a public API and Secrets Manager?

Sample answer. Public API: route 0.0.0.0/0 to a NAT gateway in a public subnet. Secrets Manager: either through NAT or an interface VPC endpoint with private DNS. For S3 and DynamoDB I use gateway endpoints. Without these routes the call times out.

INTERMEDIATE

IAM user, role, policy: how do they relate?

Sample answer. A policy is a JSON permission document. A role is an identity assumed temporarily by services or people through STS and has policies attached. Users and long-lived keys are avoided in favour of roles and SSO. Explicit deny wins, then an allow is needed, otherwise default deny.

SENIOR

Describe how you secure Lambda-to-Aurora communication end to end.

Sample answer. Private subnets, Lambda and RDS Proxy security groups chained by reference, DB with no public access; TLS enforced; credentials in Secrets Manager with rotation (or IAM authentication), fetched via VPC endpoint using a scoped execution role; KMS encryption; least-privilege DB user (no admin); CloudTrail and alarms on unusual access.

SENIOR

A function suddenly cannot read a secret after a change. How do you debug?

Sample answer. Read the exact error. AccessDenied means IAM: check the execution role policy, secret resource policy, KMS key policy for the secret key and any permission boundary or SCP. A timeout means network: check subnet route, NAT or endpoint, security groups and endpoint policy. I use CloudTrail and IAM policy simulator to confirm.

9. Scenario-based questions

SCENARIO

The security team finds the database reachable from the internet. What do you do now and later?

Strong answer. Now: remove the public IP/accessibility and the 0.0.0.0/0 rule, rotate credentials, check CloudTrail/VPC flow logs for access. Later: move it to isolated subnets, enforce with an SCP or Config rule (rds-instance-public-access-check), add GuardDuty alerts, and codify the network in CloudFormation so drift is detected.

SCENARIO

Monthly cost jumped because of NAT data processing. How do you reduce it?

Strong answer. Find the top talkers using Flow Logs/Cost Explorer. Add gateway endpoints for S3 and DynamoDB, interface endpoints for heavy AWS APIs (ECR, CloudWatch Logs, Secrets Manager), compress or cache downloads, and move traffic that does not need a VPC out of it (for example Lambdas not touching private resources).

10. Practice exercise

Exercise

Write a CloudFormation template (or draw the diagram) for a VPC with two public and two private subnets, an IGW, one NAT gateway per AZ, a gateway endpoint for S3, and security groups App → Proxy → DB. Explain which route table entry makes each subnet public or private.

Ready for live practice? Tell Claude: "Quiz me on Module 5". You will get one question at a time, feedback on your answer and follow-ups.

MODULE 6

CloudFormation and deployment

1. Simple explanation

CloudFormation lets you describe AWS resources in a YAML template and create them as a **stack**. Update the template and CloudFormation works out what to add, change or replace. **Parameters** make a template reusable (dev vs prod), **Outputs** expose values (an API URL), **DependsOn** or references (`!Ref` , `!GetAtt`) define order. A **change set** previews an update before you apply it. If something fails, the stack **rolls back** to the last known good state. **SAM** (`Transform: AWS::Serverless-2016-10-31`) is a shorthand for serverless resources that expands to plain CloudFormation.

2. Real-world analogy

> **Analogy**
>
> A template is an architect's blueprint and the stack is the building. A change set is the renovation quote: it lists which walls will be knocked down before work begins. Rollback is the contractor restoring the original structure when the renovation fails halfway. Drift detection is the inspector who finds that someone moved a wall by hand.

3. Architecture diagram

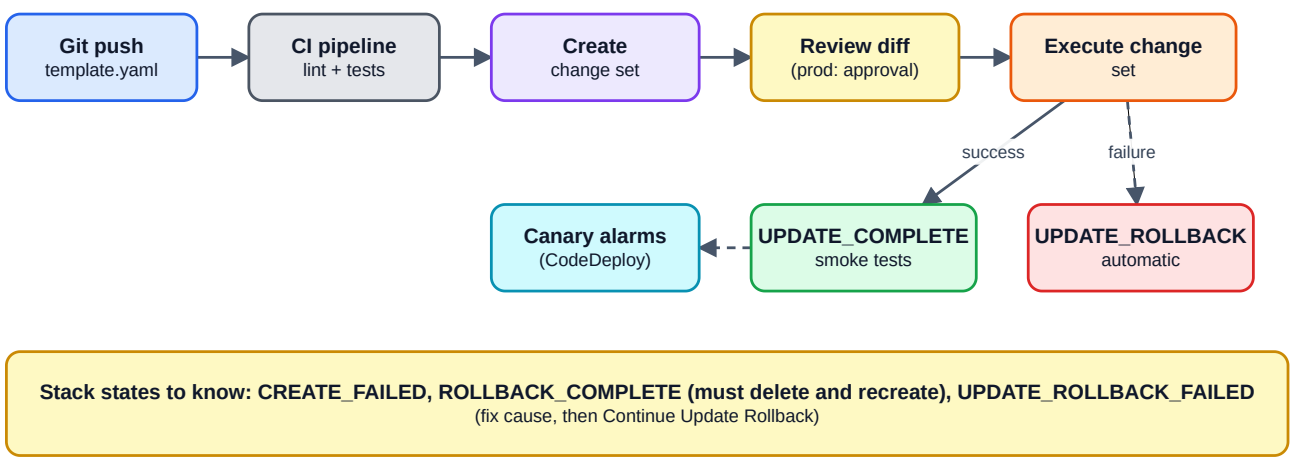


Figure 8. CloudFormation update lifecycle with change set and rollback.

4. Practical example

Environment-aware template deploying API Gateway, Lambda, SQS with DLQ and a secret reference:

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Parameters:
  Env:      { Type: String, AllowedValues: [dev, prod], Default: dev }
  DbSecretArn: { Type: AWS::SSM::Parameter::Value<String>, Default: /shop/dev/db-secret-arn }
Mappings:
  EnvCfg:
    dev:  { Memory: 256, Concurrency: 5 }
    prod: { Memory: 512, Concurrency: 100 }
Conditions:
  IsProd: !Equals [ !Ref Env, prod ]
Resources:
  OrdersDlq:
    Type: AWS::SQS::Queue
    Properties: { MessageRetentionPeriod: 1209600 }
```

```
  OrdersQueue:
    Type: AWS::SQS::Queue
    Properties:
      VisibilityTimeout: 180                    # >= 6 x function timeout (30 s)
      RedrivePolicy: { deadLetterTargetArn: !GetAtt OrdersDlq.Arn, maxReceiveCount: 3 }
  WorkerFn:
    Type: AWS::Serverless::Function
    Properties:
      Runtime: nodejs22.x
      Handler: worker.handler
      CodeUri: src/
      Timeout: 30
      MemorySize: !FindInMap [ EnvCfg, !Ref Env, Memory ]
      ReservedConcurrentExecutions: !FindInMap [ EnvCfg, !Ref Env, Concurrency ]
      AutoPublishAlias: live
      DeploymentPreference:
        Type: !If [ IsProd, Canary10Percent5Minutes, AllAtOnce ]
        Alarms: [ !Ref WorkerErrors ]
      Environment: { Variables: { DB_SECRET: !Ref DbSecretArn } }
      Policies:
        - AWSSecretsManagerGetSecretValuePolicy: { SecretArn: !Ref DbSecretArn }
      Events:
        Q: { Type: SQS, Properties: { Queue: !GetAtt OrdersQueue.Arn, BatchSize: 10, FunctionResponseTypes:
[ReportBatchItemFailures], ScalingConfig: { MaximumConcurrency: 10 } } }
  WorkerErrors:
    Type: AWS::CloudWatch::Alarm
    Properties:
      Namespace: AWS/Lambda
      MetricName: Errors
      Dimensions: [ { Name: FunctionName, Value: !Ref WorkerFn } ]
      Statistic: Sum
      Period: 60
      EvaluationPeriods: 1
      Threshold: 1
      ComparisonOperator: GreaterThanOrEqualToThreshold
Outputs:
  QueueUrl: { Value: !Ref OrdersQueue, Export: { Name: !Sub "${AWS::StackName}-QueueUrl" } }
```

```
# Safe deploy commands
aws cloudformation deploy --template-file packaged.yaml --stack-name shop-prod \
  --parameter-overrides Env=prod --capabilities CAPABILITY_IAM --no-execute-changeset   # creates a change set
aws cloudformation describe-change-set --stack-name shop-prod --change-set-name <name>  # review Replacement:
True
aws cloudformation execute-change-set  --stack-name shop-prod --change-set-name <name>
```

5. Step-by-step request flow

1. Developer opens a pull request; CI runs cfn-lint, unit tests and builds the artifact.
2. On merge, the pipeline uploads code to S3 and runs `deploy --no-execute-changeset` for production.
3. Reviewers read the change set; any resource marked Replacement: True (for example renaming a DB) is scrutinised.
4. Execute: CloudFormation updates resources in dependency order. For Lambda with DeploymentPreference, CodeDeploy shifts 10% of traffic to the new version and watches the alarm.
5. If the alarm fires or a resource fails, CloudFormation/CodeDeploy rolls back automatically.
6. Post-deploy smoke tests run; Outputs feed the frontend configuration.

6. Production considerations

- **Security:** CAPABILITY_IAM review; dynamic references (`{{resolve:secretsmanager:...}}`) instead of secrets in parameters; stack policies and termination protection on production.
- **Scalability:** split into stacks by lifecycle (network, data, app) and use nested stacks or cross-stack exports sparingly; StackSets for multi-account.
- **Reliability:** DeletionPolicy: Retain/Snapshot on databases and buckets; change sets for prod; canary + alarms; keep rollback triggers.
- **Performance:** parallelise independent resources; avoid unnecessary replacement.

- **Cost:** tag resources via stack tags; delete ephemeral stacks automatically; use drift detection to find manual changes.

7. Common mistakes

Watch out

- Changing a property that forces replacement (like a custom name or DB identifier) and losing data.
- Hard-coding names so a second environment collides.
- Circular dependencies or relying on DependsOn where a reference would do.
- Putting the database and the app in one big stack, so every app deploy risks the data layer.
- Editing resources in the console, creating drift.
- Forgetting that ROLLBACK_COMPLETE after a failed create requires deleting the stack.
- No DeletionPolicy on stateful resources.

8. Interview questions

BEGINNER

What are Parameters, Mappings, Conditions and Outputs?

Sample answer. Parameters take input values at deploy time, Mappings are static lookup tables (for example per-Region AMIs or per-environment sizes), Conditions toggle resources or properties (create something only in prod), and Outputs expose values to humans or other stacks via exports.

BEGINNER

What is a change set and why use one?

Sample answer. A preview of what an update will create, modify, or replace. I use it on production to catch replacements and data loss risks before executing.

INTERMEDIATE

What happens when a stack update fails?

Sample answer. CloudFormation rolls back to the previous state automatically. If the rollback itself fails (UPDATE_ROLLBACK_FAILED) I fix the blocking cause, such as a manually deleted resource, and continue the rollback or skip the resource. A failed create ends in ROLLBACK_COMPLETE and the stack must be deleted before retrying.

INTERMEDIATE

How do you manage dev, staging and prod?

Sample answer. The same template with parameters, mappings and conditions, deployed to separate accounts or stacks, with environment config in version-controlled parameter files. The artifact is built once and promoted. Secrets are referenced from Secrets Manager or SSM, never stored in the repo.

SENIOR

How do you do a safe Lambda release with rollback?

Sample answer. Publish a version and shift an alias using CodeDeploy canary or linear. Attach alarms on Errors, Throttles, p99 duration and business metrics, with automatic rollback. Use pre-traffic hooks for smoke tests. Database changes follow expand-and-contract so the old version still works during rollout.

SENIOR

CloudFormation vs CDK vs Terraform?

Sample answer. CloudFormation is the native engine with strong rollback and no state file to manage. CDK generates CloudFormation from real code with loops and tests, good for developers. Terraform is cloud-agnostic with a plan/apply model and a large provider ecosystem but needs state management. I choose based on team skills and multi-cloud needs; CDK/SAM suit a Node/TypeScript team on AWS.

9. Scenario-based questions

SCENARIO

A deployment failed halfway and the stack shows UPDATE_ROLLBACK_FAILED. What do you do?

Strong answer. Open the stack events to find the first failing resource. Typical causes: a resource changed or deleted manually, or a dependency not removable. Fix the cause (restore the resource or permissions) and run Continue Update Rollback, skipping stuck resources only as a last resort. Afterwards add drift detection and restrict console edits.

SCENARIO

You must rename a production DynamoDB table or RDS instance managed by CloudFormation. How?

Strong answer. Never rename in place: that triggers replacement and data loss. Take a snapshot/backup, create the new resource alongside, migrate data, switch the application, then remove the old one with a controlled change. Use DeletionPolicy Retain and import (resource import) when adopting existing resources.

10. Practice exercise

Exercise

Take the template above, deploy it twice as `Env=dev` and `Env=prod`, and use a change set to rename the queue. Observe the **Replacement** flag. Then add a CloudWatch alarm and deliberately deploy a broken handler to watch the canary roll back.

Ready for live practice? Tell Claude: "Quiz me on Module 6". You will get one question at a time, feedback on your answer and follow-ups.

MODULE 7

Storage, messaging and asynchronous processing

1. Simple explanation

S3 stores objects (files) with very high durability. **CloudFront** caches content at edge locations close to users. **SQS** is a queue: producers add messages, consumers process them later at their own pace. **SNS** is publish/subscribe: one message is pushed to many subscribers. **EventBridge** routes events by rules. An **event-driven architecture** connects services through events so they are loosely coupled. A **dead-letter queue (DLQ)** collects messages that failed too many times so you can inspect and redrive them. Most queues give **at-least-once** delivery, which means **duplicates will happen**.

2. Real-world analogy

> **Analogy**
>
> SQS is a restaurant order rail: waiters pin tickets and cooks take them when free; if a cook drops a ticket (fails), it goes back on the rail, and after three drops it goes to the manager’s tray (DLQ). SNS is the kitchen bell announcing “order up” to the waiter, the cashier and the stats board at once. S3 is the walk-in storeroom; a presigned URL is a one-time gate pass that lets a delivery driver put a box in the storeroom without getting the master key.

3. Architecture diagram

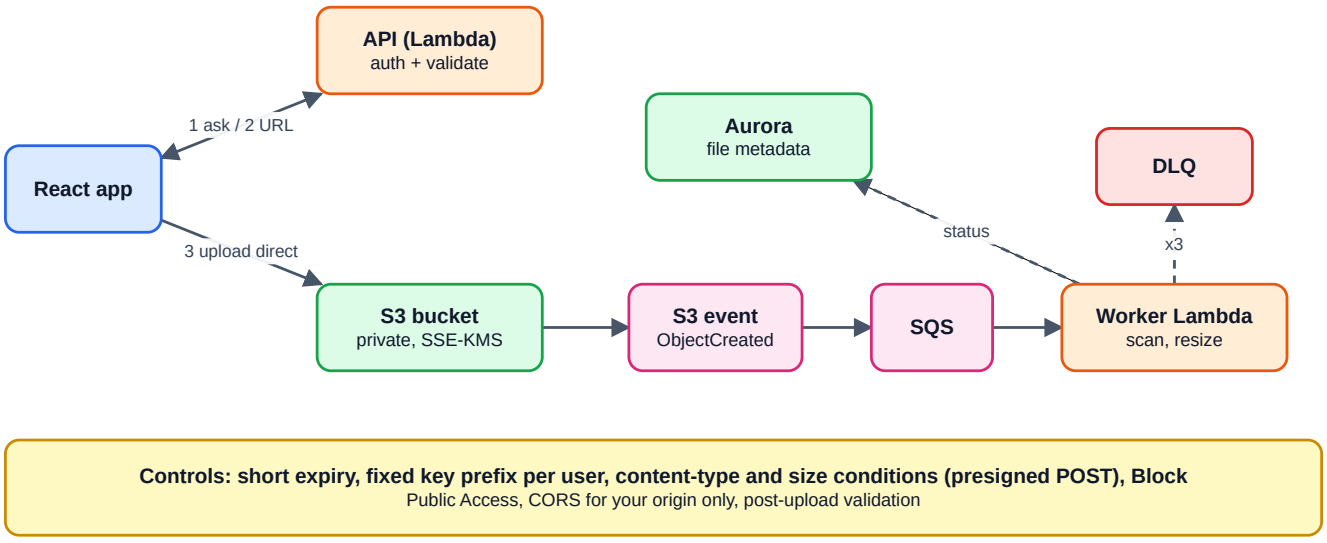


Figure 9. Secure file upload to S3 using a presigned URL, then asynchronous processing.

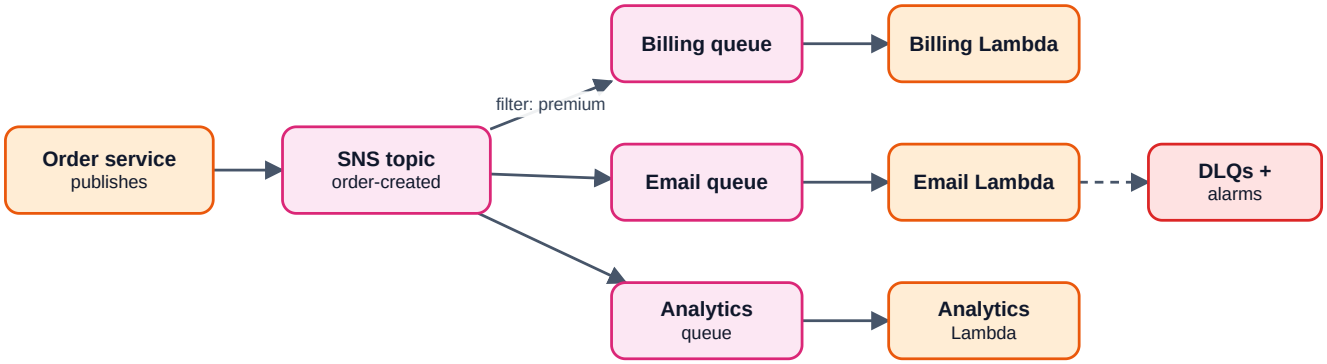


Figure 10. SNS fan-out to SQS queues, each with its own DLQ.

4. Practical example

Presigned upload (Node.js, AWS SDK v3) with enforced content type and size using a presigned POST:

```
import { S3Client } from '@aws-sdk/client-s3';
import { createPresignedPost } from '@aws-sdk/s3-presigned-post';
import { randomUUID } from 'node:crypto';
const s3 = new S3Client({});

export const handler = async (event) => {
  const userId = event.requestContext.authorizer.jwt.claims.sub;      // from the verified JWT
  const key = 'uploads/' + userId + '/' + randomUUID() + '.png';      // server chooses the key
  const post = await createPresignedPost(s3, {
    Bucket: process.env.BUCKET, Key: key, Expires: 300,
    Conditions: [ ['content-length-range', 1, 5 * 1024 * 1024], ['eq', '$Content-Type', 'image/png'] ],
    Fields: { 'Content-Type': 'image/png' }
  });
  return { statusCode: 200, body: JSON.stringify({ ...post, key }) };
};
```

```
// React: upload straight to S3 (no file bytes through your API)
const form = new FormData();
Object.entries(post.fields).forEach(([k, v]) => form.append(k, v));
form.append('file', file);                        // file field must be last
await fetch(post.url, { method: 'POST', body: form });
```

Delivery guarantees and what to do about them

Service	Delivery	Duplicates?	Ordering	Your responsibility
SQS Standard	At least once	Yes, occasionally	Best effort	Idempotent consumers, DLQ, visibility timeout
SQS FIFO	Exactly-once processing within the deduplication window	Prevented by dedup id	Per message group	Choose group ids; throughput scales with groups
SNS	At least once to each subscriber (retries per protocol)	Possible	None guaranteed (FIFO topics exist)	Fan out to SQS for durability
EventBridge	At least once	Possible	Not guaranteed	Idempotent targets, DLQ on targets, archive/replay
S3 events	At least once	Possible	Not guaranteed	Idempotent on bucket+key+versionId

Failure recovery checklist

- Alarm on **ApproximateAgeOfOldestMessage** and on DLQ depth > 0.
- Redrive from DLQ after fixing the bug (SQS console / API supports redrive to source).
- Keep DLQ retention (up to 14 days) longer than the source queue.
- Use partial batch responses so one poison message does not block nine good ones.

- Store an idempotency key (order id or message id) with a unique constraint.

5. Step-by-step request flow

1. User selects a file in React; React asks the API for an upload permission.
2. The API verifies the JWT, chooses the object key and returns a short-lived presigned POST with size and type conditions.
3. The browser uploads directly to S3 over TLS (CORS allows only your origin).
4. S3 emits an ObjectCreated event to SQS; messages wait until a consumer is ready.
5. The worker Lambda polls SQS, scans/resizes the file, stores metadata in Aurora, and deletes the message by finishing successfully.
6. On repeated failure the message moves to the DLQ and an alarm notifies the team; after the fix it is redriven.

6. Production considerations

- **Security:** private buckets with Block Public Access; CloudFront with Origin Access Control for reads; SSE-KMS; presigned URLs with short expiry; scan uploads; validate type server-side; least-privilege roles.
- **Scalability:** SQS absorbs spikes; set Lambda maximum concurrency on the event source mapping to protect downstream systems.
- **Performance:** CloudFront caching with versioned file names (hashed assets) and long TTLs; multipart upload for large files; S3 Transfer Acceleration only when proven helpful.
- **Reliability:** DLQs everywhere, idempotent consumers, S3 versioning for recovery, replication for DR.
- **Cost:** lifecycle rules to cheaper storage classes, abort incomplete multipart uploads, CloudFront reduces origin transfer cost, avoid tiny-message polling (use long polling).

7. Common mistakes

Watch out

- Assuming a message is processed exactly once and not handling duplicates.
- Making the S3 bucket public, or using one wide-open presigned URL generator without auth.
- Visibility timeout shorter than processing time.
- No DLQ, or a DLQ nobody monitors.
- Writing results to the same bucket/prefix that triggers the Lambda (infinite loop).
- Trusting the client-supplied file name or content type.
- Large messages (SQS limit is 1 MiB): store payload in S3 and pass a pointer.

8. Interview questions

BEGINNER

SQS vs SNS?

Sample answer. SQS is a pull-based queue for one consumer group to process work at its own pace. SNS is push-based pub/sub that delivers each message to many subscribers. Together (SNS to multiple SQS queues) they give fan-out with durability and independent retry.

BEGINNER

What is a dead-letter queue?

Sample answer. A separate queue for messages that failed more than maxReceiveCount times, so they stop blocking the main flow and can be inspected and redriven after a fix.

INTERMEDIATE

How do you handle duplicate messages?

Sample answer. Assume duplicates. Make the consumer idempotent with a business key and a unique constraint or conditional write, so reprocessing is a no-op. FIFO queues help within the dedup window but I still design idempotently.

INTERMEDIATE

Explain the SQS visibility timeout.

Sample answer. When a consumer receives a message it becomes invisible for the timeout. If the consumer deletes it in time, it is gone; if not, it reappears and may be processed again. Set it above processing time (for Lambda at least six times the function timeout) and extend it for long jobs.

SENIOR

Design secure file uploads to S3 for a multi-tenant app.

Sample answer. Authenticate the user, authorise the action, generate a server-chosen key under a tenant/user prefix, return a short-lived presigned POST with content-length-range and content-type conditions, upload directly to a private SSE-KMS bucket, trigger S3 event to SQS, scan for malware and validate real file type, store metadata with status, then expose downloads via short-lived presigned GET or CloudFront signed URLs. Add lifecycle, logging and alarms.

SENIOR

How do you process orders reliably with SQS and Lambda?

Sample answer. API validates and writes an order record plus an outbox/message in one transaction (or sends to SQS and returns 202), SQS buffers, Lambda consumers use batch size and maximum concurrency tuned to the database, partial batch failure responses, idempotency key = order id, DLQ with alarm and runbook for redrive, and Step Functions if steps need compensation (payment, stock, shipping).

9. Scenario-based questions

SCENARIO

Customers report missing confirmation emails. The queue looks empty. What do you check?

Strong answer. Check the DLQ and the consumer Errors metric; verify the message was actually published (producer logs); look for consumers deleting messages after swallowed errors; check SES/third-party throttling; check visibility timeout and retention. Add structured logs with message ids, a DLQ alarm and idempotent resend tooling.

SCENARIO

A bug caused 40,000 messages to land in the DLQ. How do you recover?

Strong answer. Fix and deploy the bug first (or the redrive will fail again). Verify idempotency, then redrive in controlled batches with limited concurrency to avoid hammering dependencies, monitor error rate, and report on any messages that still fail. Document the incident and add a test.

10. Practice exercise

Exercise

Implement the presigned POST flow with a React component. Reject files over 5 MB at the S3 level (try uploading 6 MB and inspect the error). Then add an S3 → SQS → Lambda processor with a DLQ and prove duplicate handling by sending the same message twice.

Ready for live practice? Tell Claude: "Quiz me on Module 7". You will get one question at a time, feedback on your answer and follow-ups.

MODULE 8

Monitoring and troubleshooting

1. Simple explanation

Monitoring answers three questions: **what happened** (logs), **how much and how fast** (metrics) and **where the time went** (traces). **CloudWatch Logs** collects output from Lambda and containers; **CloudWatch Metrics** track numbers such as Errors or Duration; **Alarms** notify or act when a metric crosses a threshold; **X-Ray** (or OpenTelemetry) traces one request across services; **CloudTrail** records who called which AWS API. Good troubleshooting is a method, not guessing: define the symptom, find when it started, compare with what changed, then narrow down the layer.

2. Real-world analogy

> **Analogy**
>
> Running production without monitoring is like driving at night without a dashboard or headlights. Logs are the black box recorder, metrics are the speedometer and fuel gauge, alarms are the warning lights, traces are the GPS route showing where the delay occurred, and CloudTrail is the CCTV showing who opened which door.

3. Architecture diagram

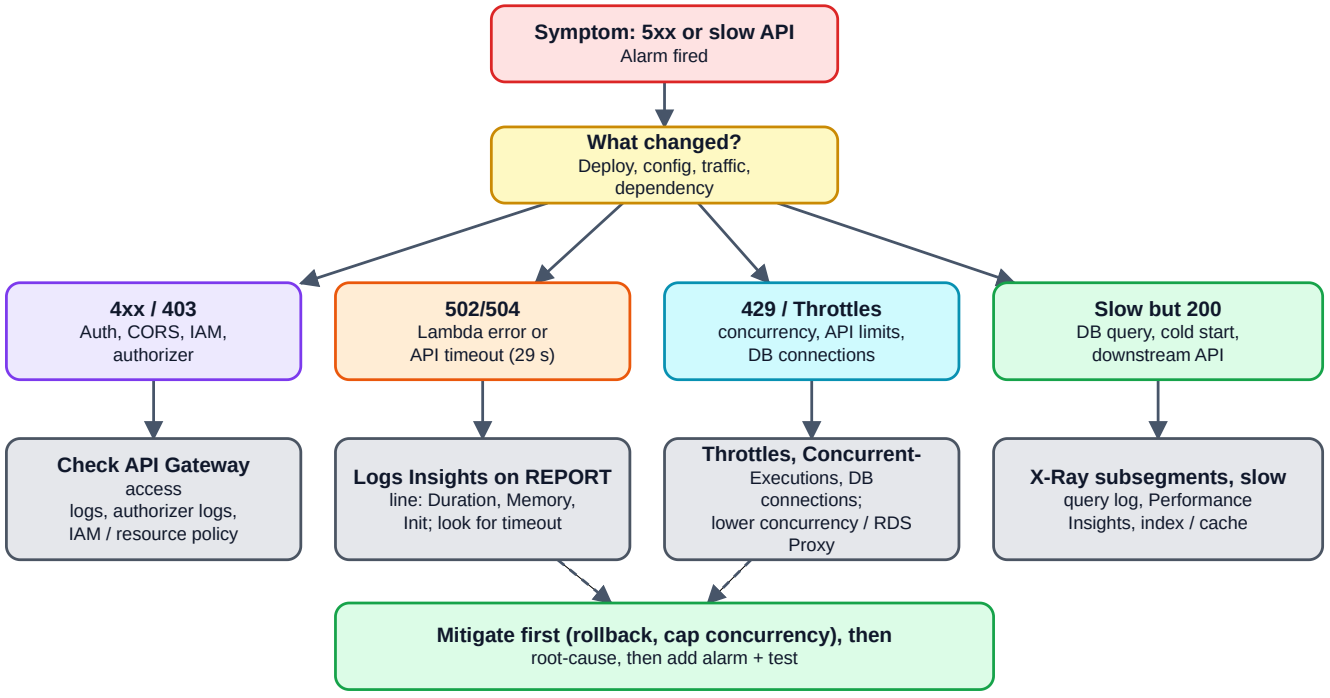


Figure 11. Production debugging decision tree for a failing API.

4. Practical example

Logs Insights queries every senior should have ready:

```
# 1. Errors by function in the last hour
fields @timestamp, @message
| filter @message like /ERROR|Task timed out/
| stats count() by bin(5m)

# 2. Slowest invocations and cold starts (Lambda REPORT lines)
filter @type = "REPORT"
| stats max(@duration) as maxMs, avg(@duration) as avgMs, max(@initDuration) as maxInit,
```

```
max(@maxMemoryUsed/1024/1024) as memMB by bin(5m)

# 3. Follow one request
fields @timestamp, @message | filter @message like /req-8f3a1c/ | sort @timestamp asc
```

```
// Structured logging that makes queries easy
const log = (level, msg, extra={}) => console.log(JSON.stringify({ level, msg, requestId: ctx.awsRequestId,
...extra }));

# Alarm (CloudFormation): p99 latency and DLQ depth
HighLatency: { Type: AWS::CloudWatch::Alarm, Properties: { Namespace: AWS/Lambda, MetricName: Duration,
ExtendedStatistic: p99, Period: 60, EvaluationPeriods: 3, Threshold: 3000, ComparisonOperator:
GreaterThanThreshold, Dimensions: [{Name: FunctionName, Value: !Ref Fn}] } }
DlqNotEmpty: { Type: AWS::CloudWatch::Alarm, Properties: { Namespace: AWS/SQS, MetricName:
ApproximateNumberOfMessagesVisible, Dimensions: [{Name: QueueName, Value: !GetAtt Dlq.QueueName}], Statistic:
Maximum, Period: 60, EvaluationPeriods: 1, Threshold: 0, ComparisonOperator: GreaterThanThreshold } }
```

Metrics to watch

Layer	Metric	Why
Lambda	Errors, Throttles, Duration p99, ConcurrentExecutions, IteratorAge (streams)	Throttles are not Errors; alarm on both
API Gateway	4XXError, 5XXError, Latency, IntegrationLatency, Count	Latency minus IntegrationLatency shows gateway overhead
SQS	ApproximateAgeOfOldestMessage, NumberOfMessagesVisible, DLQ visible	Backlog and poison messages
Aurora/RDS	CPUUtilization, DatabaseConnections, FreeableMemory, ReplicaLag, deadlocks	Connection storms and saturation
CloudFront	5xxErrorRate, CacheHitRate	Origin health and cache efficiency
Business	Orders per minute, payment success rate	Detects silent failures that infrastructure metrics miss

5. Step-by-step request flow

1. An alarm on 5XXError notifies the on-call engineer through SNS to Slack/PagerDuty.
2. Open the dashboard: is it all routes or one? When did it start? Compare with the deploy and traffic graphs.
3. In Logs Insights, filter by the failing route and group errors by message.
4. Open an X-Ray trace of a failed request to see which subsegment errored or was slow.
5. Mitigate: roll back the alias, lower concurrency, or disable a feature flag.
6. Confirm recovery in metrics, then write the root cause, fix and a new alarm or test.

6. Production considerations

- **Security:** never log secrets, tokens or full PII; restrict log access; CloudTrail to an immutable bucket.
- **Scalability:** sample traces at high volume; use metric filters or Embedded Metric Format instead of querying logs for dashboards.
- **Performance:** alarm on percentiles (p95/p99), not averages.
- **Reliability:** alarms must page a human with a runbook link; test alarms; use composite alarms to reduce noise.
- **Cost:** set log retention (the default is never expire), avoid debug logging in production, drop noisy health-check logs.

7. Common mistakes

> **Watch out**
>
> - Alerting on averages, which hide tail latency.
> - Not alarming on Throttles or DLQ depth.
> - Logging free-text strings that cannot be queried.

- Never setting log retention.
- Investigating before mitigating while customers are down.
- No correlation id across services.
- Treating CloudTrail as application logging.

8. Interview questions

BEGINNER

CloudWatch Logs vs Metrics vs Alarms?

Sample answer. Logs are the detailed text events, metrics are numeric time series, alarms evaluate a metric against a threshold and trigger actions such as SNS notifications or scaling. Logs can feed metrics through metric filters.

BEGINNER

What does X-Ray give you that logs do not?

Sample answer. A request-level view across services with timing for each segment, so I can see which downstream call made the request slow or failed, rather than correlating separate logs by hand.

INTERMEDIATE

A Lambda is erroring after a deploy. How do you respond?

Sample answer. Check whether errors began at the deploy time, read the exception in logs, and if the impact is high roll back by pointing the alias to the previous version (or let the CodeDeploy alarm roll back). Then reproduce with the failing event, fix, add a test and a canary alarm.

INTERMEDIATE

How do you tell throttling from application errors?

Sample answer. The Throttles metric and 429 responses indicate concurrency or API limits; Errors counts function failures. For SQS or streams, rising age of the oldest message or IteratorAge shows the consumer cannot keep up. I check concurrency limits, reserved concurrency and downstream capacity.

SENIOR

What would your production dashboard and alerting include?

Sample answer. Golden signals per service: traffic, errors, latency percentiles, saturation (concurrency, DB connections, CPU), plus queue age and DLQ depth, deploy markers, and a few business KPIs. Alarms page only on user-impacting symptoms with runbooks; others create tickets.

SENIOR

A deployment succeeded but users report intermittent failures. How do you find the cause?

Sample answer. Segment by version/alias, region, user and endpoint; compare error rates between old and new versions (canary); look at traces for failures; check dependency changes, config and secrets rotations, cold starts, and database limits. Use feature flags to isolate and roll back quickly if needed.

9. Scenario-based questions

SCENARIO

The API is slow only between 9:00 and 9:15 every morning. How do you investigate?

Strong answer. Correlate with traffic and scheduled jobs: batch reports, cache expiry, Lambda cold starts from a spike, database CPU and connections. Use X-Ray and Performance Insights for that window, check cron jobs, and fix by caching, pre-warming with provisioned concurrency, moving heavy jobs off-peak or isolating them on a replica.

SCENARIO

You see 5xx at API Gateway but no Lambda errors. Where could the problem be?

Strong answer. Possibilities: integration timeout (29 s), Lambda throttled (429 mapped to 5xx), response format invalid for proxy integration (502 Malformed Lambda proxy response), payload too large, authorizer failures, or permission problems invoking the function. Enable API Gateway execution/access logs to see integration status and latency.

10. Practice exercise

Exercise

Instrument one Lambda with structured JSON logs, Powertools or a custom correlation id, an alarm on p99 Duration and on Throttles, and X-Ray active tracing. Introduce a 5-second sleep and an unhandled exception and write the exact Logs Insights queries and alarm notifications you would use to find each.

Ready for live practice? Tell Claude: "Quiz me on Module 8". You will get one question at a time, feedback on your answer and follow-ups.

MODULE 9

System design and real-world scenarios

1. Simple explanation

System-design questions test whether you can turn requirements into a working, resilient architecture and explain trade-offs. A reliable structure is: **1) clarify requirements** (users, traffic, latency, consistency, compliance, budget), **2) draw the happy path**, **3) identify bottlenecks and failure modes**, **4) add scaling, security and observability**, **5) state trade-offs and what you would measure**. Always find the stateful bottleneck: it is usually the database.

2. Real-world analogy

Analogy

Designing a system is city planning: roads (network), power (compute), warehouses (storage), post office (queues), police and fire (security and monitoring). A good planner asks how many people will live here and what happens during a festival or a blackout before drawing any roads.

3. Architecture diagram

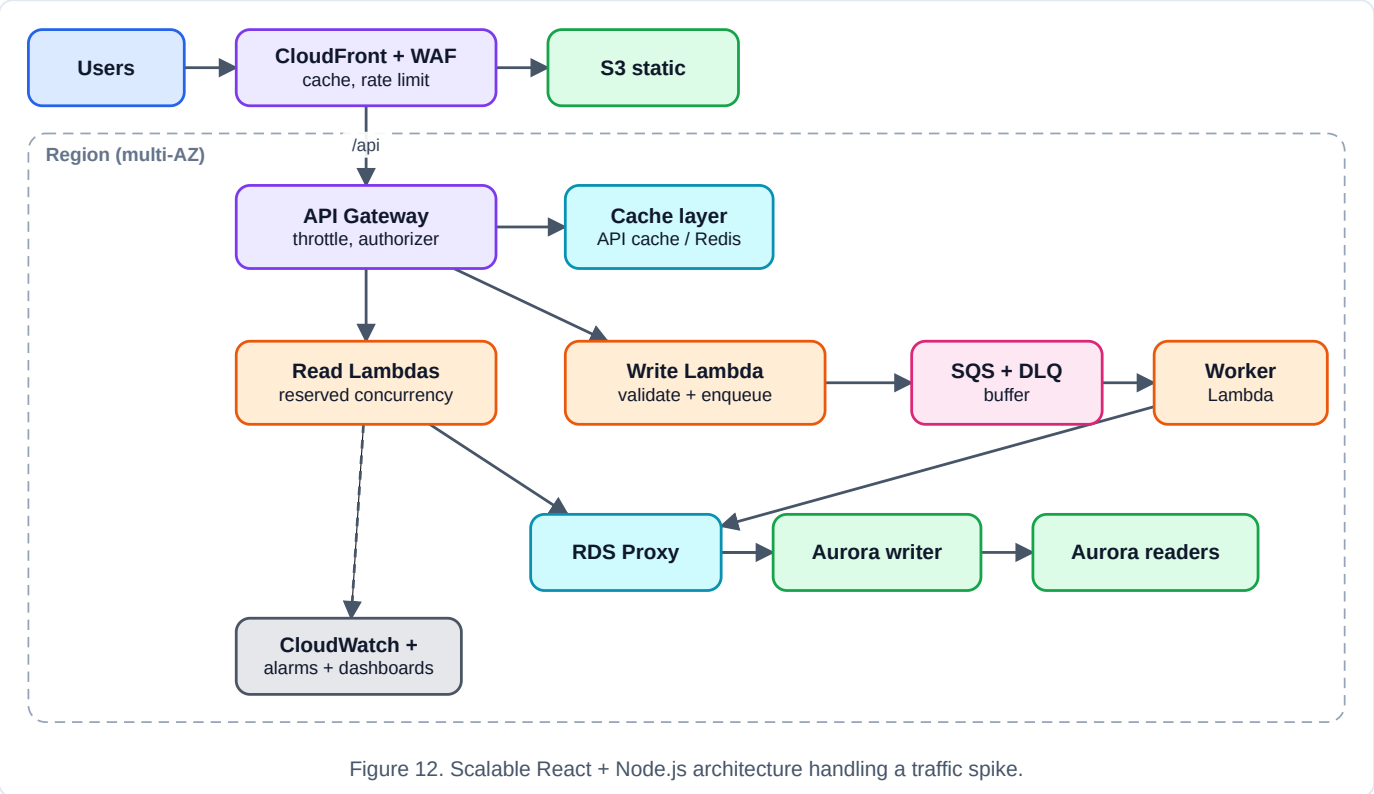


Figure 12. Scalable React + Node.js architecture handling a traffic spike.

4. Practical example

The seven scenarios you should be able to whiteboard (short answers):

Scenario	Answer skeleton
Design a scalable React + Node.js app on AWS	S3+CloudFront for the SPA; Cognito; API Gateway/AppSync to Lambda (spiky) or ECS Fargate (steady); Aurora MySQL with RDS Proxy and readers; SQS for background work; Secrets Manager; CloudWatch/X-Ray; CloudFormation + CI/CD; two AZs.
Sudden increase in API traffic	Edge: CloudFront caching, WAF rate rules. API: throttling, caching GETs. Compute: Lambda scales, but cap concurrency to protect the DB; move writes behind SQS; scale Aurora readers or Serverless v2; shed load gracefully with 429 + Retry-After; alarms and a load-test plan in advance.

Lambda opens too many MySQL connections	Pool outside handler with limit 1 to 2; RDS Proxy; reserved concurrency / SQS maximum concurrency; shorten transactions; cache; alarm on DatabaseConnections.
Process orders reliably with SQS + Lambda	Validate, persist, enqueue; idempotency key; batch + partial failures; visibility timeout >= 6x function timeout; DLQ + alarm + redrive; Step Functions saga for payment/stock/shipping compensation.
Secure file uploads to S3	Auth, server-chosen key, presigned POST with size/type conditions, private SSE-KMS bucket, event to SQS, scan, short-lived download links, lifecycle, logging.
Deploy safely and roll back	CloudFormation change sets, immutable versions, alias canary with alarms, expand-contract DB migrations, feature flags, automated rollback and runbook.
Optimise a slow GraphQL API on Aurora	Trace resolvers; fix N+1 with batching; indexes and EXPLAIN; select needed columns; keyset pagination; replicas; caching; depth/complexity limits; denormalised read model if needed.

Trade-off phrases that sound senior

- “Lambda gives me elasticity, but the database is the real scaling limit, so I would cap concurrency and buffer writes with SQS.”
- “I would start simple (managed services, single Region, two AZs) and add complexity only when metrics justify it.”
- “For consistency-critical flows like payments I prefer a transactional database plus idempotency; for feeds I accept eventual consistency and caching.”
- “Define RPO and RTO with the business first; the DR design follows from them.”
- “I would load-test and set alarms before the launch, not after.”

5. Step-by-step request flow

1. Clarify: expected peak requests per second, read/write ratio, latency target, data size, compliance, budget.
2. Estimate: 2,000 RPS x 150 ms = about 300 concurrent Lambda executions; check quotas and DB capacity.
3. Edge: CloudFront serves static content and caches cacheable API GETs; WAF filters abuse.
4. API layer authenticates and throttles; reads go to Lambda/readers, writes validate and enqueue.
5. Workers process queue messages at a controlled rate through RDS Proxy into Aurora.
6. Observe: dashboards and alarms for latency, errors, queue age, DB connections; autoscaling and runbooks cover failure.

6. Production considerations

- **Security:** least privilege, private data tier, WAF, secrets rotation, encryption, audit logs.
- **Scalability:** scale stateless tiers horizontally, buffer with queues, scale reads with replicas and caches, plan write scaling.
- **Performance:** cache at the edge, keep hot paths short, index, avoid N+1, use connection pooling.
- **Reliability:** multi-AZ, retries with backoff and jitter, DLQs, backups and restore drills, graceful degradation.
- **Cost:** estimate monthly cost early; know the three biggest cost drivers; use budgets and anomaly detection.

7. Common mistakes

Watch out

- Jumping to services before clarifying requirements.
- Designing multi-Region active-active when the business needs a simple DR plan.
- Forgetting the database bottleneck, or ignoring cache invalidation.
- Ignoring failure modes, security and observability.
- Using too many services to look impressive.
- Not giving numbers: capacity estimates show maturity.
- Not stating trade-offs.

8. Interview questions

BEGINNER

How do you start a system design answer?

Sample answer. I clarify requirements and constraints first (traffic, latency, data, compliance, budget), then sketch the simplest architecture that meets them and extend it where I find bottlenecks or risks.

INTERMEDIATE

How do you estimate Lambda concurrency and database load?

Sample answer. Concurrency equals requests per second times average duration in seconds. If I expect 1,000 RPS at 200 ms, that is about 200 concurrent executions. I then check the account quota and what database connections and queries per second that implies, and design caps, pooling and caching accordingly.

INTERMEDIATE

Synchronous vs asynchronous APIs?

Sample answer. Synchronous suits quick user-facing requests that need an immediate answer. Asynchronous (202 plus queue/worker, then polling or push) suits slow, spiky or failure-prone work, improving resilience and smoothing load at the cost of eventual consistency and extra complexity.

SENIOR

How do you handle a sudden 10x traffic increase?

Sample answer. Mitigate at the edge with cache and WAF rate limits, shed load with 429s, put writes behind SQS, increase Aurora reader capacity or ACU limits, protect the DB with reserved concurrency, scale ECS tasks if used, and watch saturation signals. Afterwards load-test, set autoscaling and alarms, and fix the root bottleneck.

SENIOR

What would you change when the product becomes global?

Sample answer. CloudFront already helps for static and cacheable content. For dynamic data, consider Route 53 latency routing to multiple Regions, Aurora Global Database or per-Region data ownership, DynamoDB global tables for suitable data, regional queues, and a clear consistency and failover strategy. I would only do it with a real latency or DR requirement because it increases complexity.

SENIOR

How do you balance cost and reliability?

Sample answer. I express reliability as SLOs and error budgets, spend on redundancy where the business impact is highest (data, payments), and use cheaper patterns elsewhere (scale to zero, Graviton, Savings Plans, lifecycle). Measure cost per request or tenant and review monthly.

9. Scenario-based questions

SCENARIO

Design a flash-sale checkout for 50,000 users in five minutes.

Strong answer. Pre-warm: provisioned concurrency and Aurora readers; CloudFront caches the catalogue; waiting-room or rate limiting at the edge; checkout API validates and writes an order intent to SQS FIFO per product or does a conditional stock decrement; idempotency keys; workers handle payment via Step Functions saga; inventory atomic updates in SQL; DLQ and dashboards; load test beforehand; graceful “sold out” responses.

SCENARIO

Your monolithic Express API runs on one EC2 instance. How do you migrate to AWS-native safely?

Strong answer. Containerise and run on ECS Fargate behind an ALB first (lift and improve), put the DB on Aurora with backups, move static assets to S3/CloudFront, externalise sessions and config, add CI/CD and monitoring. Then extract bursty or async features into Lambda/SQS gradually with the strangler pattern, shifting traffic with weighted routing.

10. Practice exercise

Exercise

Pick one scenario (flash sale or file-upload platform). In 10 minutes, speak aloud and draw: requirements, numbers, architecture, three failure modes, scaling plan, security controls, monitoring and cost. Record yourself and check that you stated at least two trade-offs.

Ready for live practice? Tell Claude: "Quiz me on Module 9". You will get one question at a time, feedback on your answer and follow-ups.

APPENDIX

Cheat sheets and progress tracker

Key numbers (confirm in Service Quotas)

Service	Fact to remember
Lambda	15 min timeout; 128 MB to 10,240 MB memory; 6 MB sync payload; 1,000 default concurrency per Region; async: 2 retries, up to 6 h event age; env vars 4 KB
API Gateway	About 29 s default integration timeout for REST; HTTP API 30 s; payload 10 MB; cache TTL default 300 s (REST only)
SQS	Message up to 1 MiB; retention 1 min to 14 days (default 4 days); visibility timeout default 30 s, max 12 h; long polling up to 20 s
S3	Object up to 5 TB; single PUT up to 5 GB; strongly consistent; presigned URL expiry set by you
Aurora MySQL	6 copies across 3 AZs; up to 15 readers; storage up to 128 TiB; backups retention up to 35 days
VPC	Security group stateful allow-only; NACL stateless allow/deny; one NAT gateway per AZ for HA
CloudFormation	Use change sets in prod; ROLLBACK_COMPLETE needs delete; DeletionPolicy for stateful resources

Glossary

Term	Meaning
Idempotent	Safe to run more than once with the same result
Execution environment	The sandbox in which a Lambda function runs; handles one request at a time
ESM	Event source mapping: Lambda-managed poller for SQS, Kinesis, DynamoDB Streams
DLQ	Dead-letter queue for messages that repeatedly fail
RDS Proxy	Managed connection pooler for RDS and Aurora
ENI	Elastic network interface; how VPC resources attach to subnets
OAC	Origin Access Control: lets CloudFront read a private S3 bucket
RPO / RTO	Maximum acceptable data loss / maximum acceptable downtime
Change set	Preview of CloudFormation changes before execution
Canary deployment	Send a small share of traffic to the new version first

Weak-topic tracker

After each live quiz question, mark your confidence. Revisit anything rated 1 or 2 within three days.

Module	Topic I struggled with	Score 1 to 5	Revisit date	Done
1				
2				
3				
4				
5				
6				
7				
8				
9				

Final interview checklist

- I can explain sync vs async vs ESM retries without notes.
- I can draw a VPC with public, private and data subnets and explain every route.
- I can fix “Lambda opens too many MySQL connections” in three ways.
- I can describe a safe release and rollback with CloudFormation and CodeDeploy.
- I can design secure S3 uploads and reliable SQS order processing.
- I can debug a production incident with a clear method and say what I would alarm on.
- I always state trade-offs and numbers.